Computers & Industrial Engineering 210 (2025) 111525



Contents lists available at ScienceDirect

Computers & Industrial Engineering

journal homepage: www.elsevier.com/locate/caie



Semantic-driven real-time scheduling approach for aerospace product manufacturing

Chen Ding [a], Fei Qiao [a], Juan Liu [a,*], Dongyuan Wang [a], Jiaxuan Shi [b]

[a] School of Electronics and Information Engineering, Tongji University, Shanghai, China
[b] Shanghai Research Institute for Intelligent Autonomous Systems, Tongji University, Shanghai, China

ARTICLE INFO

Keywords:
Aerospace product manufacturing
Real-time scheduling
Knowledge graph
Deep reinforcement learning

ABSTRACT

In aerospace product manufacturing (APM), scheduling faces significant challenges due to the high uncertainty of disturbances, the increasing demand for customized products, and the complexity of managing heterogeneous data from multiple sources. To address these challenges, this study presents a semantic-driven real-time scheduling approach (SRSA) that integrates knowledge graph with deep reinforcement learning. A knowledge graph model (KGM) is developed to provide a unified semantic representation of product, process, and resource information, facilitating seamless integration of diverse data in the workshop. During the scheduling process, the KGM can provide workshop status information to support the generation of scheduling decisions. Moreover, the logical rule-based response strategy is embedded in the KGM to enhance disturbance handling. A multi-agent soft-actorcritic (MASAC) method is developed, with carefully designed state spaces, action spaces, reward functions, and asynchronous transitions. Agents are trained to dynamically select appropriate dispatching rules based on real-time information provided by the KGM. The main contributions include the integration of knowledge graphs for semantic modeling of the APM workshop, the design of logical response rules for disturbance mitigation, and the development of a MASAC for efficient scheduling. A case study in an APM workshop demonstrates the effectiveness of the proposed SRSA. The results show that SRSA significantly improves decision-making capabilities and outperforms existing methods in a highly dynamic environment.

1. Introduction

With the rapid development of advanced manufacturing technologies, the market demand for customized products in aerospace manufacturing has increased significantly. At the same time, the manufacturing mode for aerospace products has shifted from mass production to small-scale customized production (Jan et al., 2023). In this context, aerospace manufacturing orders typically have the characteristics of small quantities and high variety. Each product within an order has unique processing requirements, which adds considerable uncertainty and complexity to the production process. To accommodate customized production, manufacturing workshops equipped with advanced, multifunctional machines have become increasingly prevalent in aerospace product manufacturing (APM).

In APM, the high levels of uncertainty, dynamics, and complexity inherent to the production environment result in frequent and unpredictable disturbances, such as machine failures, emergency orders, and random product arrivals. These disturbances necessitate the real-time adjustment of scheduling decisions. Furthermore, the processes and resources required for different products are complex and diverse, reflecting the nature of customized production (Li & Tang, 2021; Gu et al., 2024). The frequency of information updates often lags the need for production schedule adjustments, posing significant challenges to scheduling. Therefore, the dynamic scheduling in the APM workshop can be decomposed into two key subproblems, namely, (1) *how to quickly acquire information about products and resources to promptly respond to scheduling demands when disturbance events occur*, and (2) *how to make real-time scheduling decisions to ensure optimal resource allocation and operation sequencing.*

To solve the scheduling problem in APM, a semantic-driven real-time scheduling approach (SRSA) is proposed by combining the knowledge graph and multi-agent soft actor-critic (MASAC). The motivations are as follows. First, the complexity and variability of APM make it essential to efficiently collect, interpret, and apply vast amounts of production information. Traditional scheduling methods often struggle to handle the diverse and dynamic data associated with customized production,

* Corresponding author at: School of Electronics and Information Engineering, Tongji University No.4800 Cao'an Road, Shanghai 201804, China.
E-mail address: juanliu0720@tongji.edu.cn (J. Liu).

https://doi.org/10.1016/j.cie.2025.111525
Received 14 September 2024; Received in revised form 3 August 2025; Accepted 5 September 2025
Available online 7 September 2025
0360-8352/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

resulting in delays and inefficiencies in responding to disturbance events. To address these issues, a knowledge graph model (KGM) has been developed for the APM workshop. It provides a structured and semantically rich representation of the production environment. By integrating KGM into production scheduling, systems can better understand and exploit the complex relationships between products, resources, and processes. Second, although deep reinforcement learning (DRL) methods have the strength to make adaptive scheduling decisions in real-time, they often lack the ability to incorporate and reason about the rich information that is crucial for understanding complex manufacturing processes. By combining the reasoning capabilities of knowledge graphs with the adaptive decision-making strengths of DRL, it is possible to create a more powerful and context-aware scheduling system. This collaborative approach allows DRL agents to leverage the deep semantic insights provided by knowledge graphs to make informed decisions based on a comprehensive understanding of the production environment. The goal of this study is to provide a more intelligent and adaptive solution for managing the complexities of dynamic scheduling in aerospace manufacturing. The main contributions of this study are as follows.

(1) A semantic-driven real-time scheduling approach is proposed to solve the APM scheduling problem. A KGM is constructed to integrate product, process, and resource information, enabling a unified representation of the workshop environment.
(2) A logical rule-based response strategy is embedded in the KGM to enhance disturbance handling in dynamic environments. Meanwhile, the KGM continuously provides real-time workshop status to support adaptive and responsive decision-making.
(3) A MASAC method is developed and implemented in SRSA. The state spaces, action spaces, reward functions, and asynchronous transitions are meticulously designed to capture the complexities of the scheduling task.

The remainder of this paper is organized as follows. Section 2 provides an overview of relevant research. Section 3 provides a detailed description of the APM scheduling problem. In Section 4, the proposed SRSA is elaborated. Section 5 contains a case study that demonstrates the effectiveness of the proposed approach. Finally, Section 6 concludes this paper.

2. Literature review

This section reviews recent advances related to two key areas: the application of knowledge graphs in manufacturing and dynamic scheduling methods under uncertainty. Additionally, it highlights the research gaps that motivate this study.

2.1. Knowledge graph in manufacturing

The concept of the knowledge graph was introduced by Google in 2012. Since then, knowledge graphs have gained significant attention in both academia and industry for their ability to integrate heterogeneous data in a semantic and machine-readable format. Knowledge graph employs an ontology as a formal framework to define classes, relations, and attributes, thereby offering a clear and unambiguous understanding of a domain. Semantic information is encoded using "entity–relation–entity" triples, in which entities are represented as nodes and relationships as edges, forming an interconnected network of knowledge. This structure enables the explicit mapping of complex semantic relationships, making it easier to query and derive valuable insights for decision-making (Lim et al., 2024; Guan et al., 2023). As semantic networks, knowledge graphs can capture causality, dependencies, and physical interconnections among knowledge entities, endowing them with strong expressive power and modeling flexibility.

In the context of intelligent manufacturing, knowledge graphs have been widely adopted to address challenges related to data heterogeneity and knowledge reuse. They facilitate the semantic integration of multi-source data, originating from enterprise systems and Internet of Things (IoT) devices, thus enhancing interoperability across different layers of manufacturing systems. Recent studies have explored the application of knowledge graphs in diverse manufacturing scenarios, as shown in Table 1. In surface defect recognition, knowledge graphs act as semantic bridges to transfer relational knowledge between the base and novel defect classes (Li et al., 2022b). In product design, they enable the structured representation and integration of complex design knowledge, facilitating intuitive exploration and effective ideation to support the development of innovative products (Cong et al., 2025). In assembly process planning, knowledge graphs support the extraction, integration, and utilization of complex assembly knowledge, thereby facilitating automated and efficient assembly process planning (Xiao et al., 2024). In production logistics, the knowledge graph enables the reasoning of dynamic spatial–temporal logistics data, supporting efficient and intelligent resource allocation (Zhao et al., 2022). In product quality traceability, the knowledge graph enables intelligent analysis and reasoning to enhance the quality assurance of products throughout their lifecycle (Kong et al., 2024). Furthermore, in fault diagnosis, knowledge graphs provide a structured representation that integrates and links multimodal information, thereby enhancing diagnostic accuracy and interpretability (Peng et al., 2024).

Table 1
Application of knowledge graph in manufacturing scenarios.

Application domain	Representative work	Use of knowledge graph	Innovative aspects
Surface defect recognition	Li et al., 2022b	Transfer semantic relationships between defect classes	Improves generalization for novel defect recognition
Product design	Cong et al. 2025	Structured integration of design knowledge	Supports intuitive exploration and innovative product development
Assembly process planning	Xiao et al. 2024	Integration of assembly process knowledge	Enables automated and efficient planning
Production logistics	Zhao et al. 2022	Reasoning over dynamic spatial–temporal logistics data	Enhances intelligent resource allocation
Product quality traceability	Kong et al. 2024	Integration and reasoning of lifecycle quality data	Improves quality assurance and traceability
Fault diagnosis	Peng et al. 2024	Linking and integration of multimodal fault information	Improves diagnostic accuracy and interpretability

Collectively, these developments demonstrate that knowledge graphs serve not only as foundational tools for semantic integration, but also as key enablers for building intelligent, efficient, and explainable manufacturing systems.

2.2. Dynamic scheduling

In the past decade, many scholars have presented various dynamic scheduling methods to cope with disturbance events. These methods are generally classified into three categories (Dauzère-Pérès et al., 2023): robust scheduling, predictive-reactive scheduling, and real-time scheduling. Robust scheduling considers a range of uncertainties and attempts to increase the robustness of the scheduling scheme through proactive measures. However, it often lacks the flexibility and adaptability needed in highly dynamic environments where disturbances occur frequently (Wu & Yan, 2023). Predictive-reactive scheduling generates a scheduling scheme in advance, which is then adjusted during production in response to dynamic disturbance events (Destouet et al., 2023; Qiu et al.,

Table 2
An overview of real-time scheduling methods.

Type	Recent work	Method	Objective	Disturbance	Description
Heuristic rule	Oukil & El-Bouri, 2021	VATC	Makespan, mean flowtime, & mean tardiness	Arrival of new job order	Easy to implement, but lacks adaptability to dynamic environments.
	Xiong et al., 2024	WLS	Weighted least slack	Random job arrivals	
	Salama et al., 2022	GP	Total weighted tardiness, mean tardiness, & mean flow time	Random job arrivals	
	Chen et al., 2025	GP	Makespan & total tardiness	New job insertions	
RL	Wang, 2020	Q-learning	Total earliness & tardiness	Random job arrivals	Classical RL: efficient for low-dimensional scheduling problems, but suffers from state explosion problem.
	Lavanya & Saravanan, 2025	SARSA	Makespan & energy consumption	Workload fluctuations	
	Luo, 2020	DQN	Total tardiness	New job insertions	DRL: excels at handling high-dimensional and dynamic scheduling tasks with strong generalization, but requires extensive training data.
	Li et al., 2022a	HDQN	Makespan & total energy consumption	New job insertions & machine breakdowns	
	Lu et al., 2024	DDQN	The sum of makespan and mean tardiness	Dynamic job arrivals & urgent job insertions	
	Wu et al., 2024	PPO	Makespan	Uncertain processing time	
	Gui et al., 2023	DDPG	Mean tardiness	Random job arrivals	
	Zhang et al., 2024	D3QN	Weighted tardiness	Random job arrivals	

2024). This approach relies heavily on accurate predictions of future conditions, making it vulnerable to prediction errors. Real-time scheduling, also known as completely reactive scheduling, responds directly to the current production state or disturbance events without the need for a pre-generated schedule. This method excels in real-time performance, making it particularly effective in dynamic environments. Compared to the other two methods, real-time scheduling offers superior real-time response and flexibility, better meeting the demands of environments where rapid decision-making is required.

The mainstream real-time scheduling methods include heuristic rules and reinforcement learning (RL) methods, as shown in Table 2. Heuristic rules are easy to implement and highly flexible. Classical rules, such as earliest due date (EDD), shortest processing time (SPT), and first in first out (FIFO), smallest queue size (SQ), are widely used. In recent years, several improved heuristic rules have been proposed, including variants of the apparent tardiness cost (VATC) (Oukil & El-Bouri, 2021) and the weighted least slack (WLS) (Xiong et al., 2024). Moreover, genetic programming (GP) has emerged as a promising approach for generating efficient dispatching rules (Salama et al., 2022; Chen et al., 2025). These rules can respond quickly to disturbances, but no single rule can adapt to all situations in a dynamic environment (Zhang et al., 2023). RL methods, on the other hand, model the dynamic scheduling process as a Markov decision process (MDP). They aim to maximize the expected sum of long-term rewards by interacting with the environment. Differing from dispatching rules, RL agents can adaptively choose the most appropriate action based on the observed state at each decision point (Luo, Zhang, & Fan, 2021).

In the past decade, classical RL methods such as Q-learning (Wang, 2020) and SARSA (Lavanya & Saravanan, 2025) have been studied to solve production scheduling problems. While these methods can deliver satisfactory performance for certain scheduling tasks, they often face the state explosion problem, particularly in production scenarios where the state space is continuous. Maintaining a single lookup Q-table for all state-action pairs becomes infeasible in such cases. Deep reinforcement learning (DRL) methods can mitigate this problem by utilizing deep neural networks as function approximators (Ladosz et al., 2022). By replacing Q-tables with function approximators, DRL methods can handle continuous state spaces effectively. DRL methods such as deep Q network (DQN) (Luo, 2020), hybrid DQN (HDQN) (Li et al., 2022a), double deep Q network (DDQN) (Lu et al., 2024), proximal policy optimization (PPO) (Wu et al., 2024), deep deterministic policy gradient (DDPG) (Gui et al., 2023), dueling double deep Q-network (D3QN) (Zhang et al., 2024), have been successfully applied to dynamic scheduling problems. These methods show that dynamically selecting the most suitable rule from a set of candidate rules at decision points can lead to better performance than using a single rule. This ability allows them to achieve both long-term scheduling performance and real-time adaptability.

2.3. Research gaps

From the above literature review, despite the increasing application of knowledge graphs in manufacturing and advances in dynamic scheduling methods, several research gaps remain.

First, while knowledge graphs have been widely used for data integration and semantic representation in intelligent manufacturing, their application to production scheduling has received limited attention. A unified framework that semantically models product-process-resource relationships and dynamic disturbances for production scheduling is still lacking.

Second, while DRL has shown promise in dynamic scheduling, it typically operates with limited contextual awareness. Existing studies rarely integrate semantic reasoning from knowledge graphs with the adaptive decision-making of DRL agents. This lack of integration restricts the interpretability and responsiveness of scheduling agents, particularly in the face of diverse disturbances and evolving production conditions.

Therefore, this study proposes the SRSA that combines the knowledge graph and DRL. A KGM is developed to provide a unified representation of the workshop environment and to support real-time scheduling decision-making in complex production environments. By combining the reasoning capabilities of knowledge graphs with the adaptive decision-making strengths of DRL, the proposed approach enhances the context-awareness and disturbance-resilience of scheduling systems in APM.

3. Problem description

The APM scheduling problem with dynamic disturbances in a flexible workshop is described as follows. The workshop consists of n workstations and u machines, where the set of workstation nodes is denoted as $W = \{W_i, i = 1, \cdots, n\}$, and the set of machine nodes is denoted as $M = \{M_1, M_2, \cdots, M_u\}$. Each workstation node contains a set of machines, and each machine belongs to only one workstation. The products to be processed are represented by $P = \{P_j, j = 1, \cdots, l\}$, where the operations for the j-th product are denoted as $O_j = \{O_{jk}, k = 1, \cdots, q\}$. Each operation O_{jk} is performed on a machine M_s from a set of available machines M_{id} ($M_s \in M_{id}, M_{id} \subseteq W_i$) at the workstation W_i. Each product, following its corresponding process route, is processed on different machines across various workstations.

Table 3
Parameter notations.

Notation	Descriptions
C_j	the completion time of product P_j
D_j	the due date of product P_j
A_j	the arrival time of product P_j
C_{jks}	the completion time of the operation O_{jk} on machine M_s
S_{jks}	the start time of the operation O_{jk} on machine M_s
t_{jks}	the operation time of the operation O_{jk} on machine M_s
$C_{j(k-1)s'}$	the completion time of the operation $O_{j(k-1)}$ on machine $M_{s'}$
$S_{j'k's}$	the start time of the operation $O_{j'k'}$ on machine M_s
$C_{j'k's}$	the completion time of the operation $O_{j'k'}$ on machine M_s
X_{jks}	it is equal to 1 when the operation O_{jk} is assigned to the machine M_s; otherwise, it is equal to 0
FT_{rs}	the start time of the r-th failure on machine M_s
rt_{rs}	the repair time of the r-th failure on machine M_s
AV_s	the available time of machine M_s
n	the number of workstations
u	the number of machines
T_{sum}	the total tardiness of all products

In addition, three most common disturbances in APM workshop are considered, which are described as follows. a) Machine failure occurs when a machine unexpectedly stops working, causing disruptions in the manufacturing process and requiring immediate repair to keep the machine running properly. b) Emergency orders are characterized by urgent manufacturing requests with tight due date. These orders are prioritized over other production tasks, often requiring adjustments to the existing production schedule to meet critical deadlines. c) Random product arrivals refer to the introduction of new products into the production workshop. Unlike emergency orders, these arrivals do not have urgent due dates and are scheduled based on existing production capacity and resource availability.

To ensure a clear understanding of the problem, the following statements are provided:

(1) In APM, new products, including emergency orders, arrive randomly at the workshop according to a certain probability distribution.
(2) New products arrive at any time and are decomposed into a set of operations. New arrivals are not necessarily processed immediately if the current production status does not allow it.
(3) Products refer to machined products and do not involve assembly, which means that each product consists of a single entity.
(4) If the previous product is not finished on the machine, the next product must wait. Each product cannot be processed on more than one machine at the same time.
(5) Each machine cannot process multiple operations at the same time, and each operation follows a specific processing sequence.
(6) If a machine fails, the operation of the product being processed is interrupted. The machine is repaired immediately, with a specific repair time. Once the repair is complete, the interrupted operation resumes.
(7) All products in the workshop are assigned one of two processing priorities: high or low. Products from emergency orders are given a high priority, while all other randomly arriving products are assigned a low priority.

The notations used in this study are summarized in Table 3.

The scheduling task involves machine assignment (MA) and operation sequencing (OS) in a highly dynamic environment. An illustration of MA and OS is shown in Fig. 1. Specifically, MA refers to the allocation of which machine will process a specific product, while OS involves arranging the order in which products are executed on the machine. The primary objective is to minimize total tardiness, a critical criterion in APM. Achieving this goal can enhance operational efficiency, reduce costs, and maintain high levels of customer satisfaction. Additionally, minimizing tardiness ensures a stable supply chain by delivering products on schedule, preventing bottlenecks and disruptions that could impact the entire aerospace production line. Based on the above descriptions, the considered scheduling problem is formulated as follows.

$$Min: T_{sum} = \sum_{j=1}^{l} \max(C_j - D_j, 0) \quad (1).$$

s.t.

$$C_{jks} > S_{jks} \quad (2)$$

$$\sum_{s=1}^{u} X_{jks} = 1 \quad (3)$$

$$S_{jks} > C_{j(k-1)s'} \quad (4)$$

$$(C_{jks} - t_{jks} - C_{j(k-1)s'})X_{jks} \geq 0 \quad (5)$$

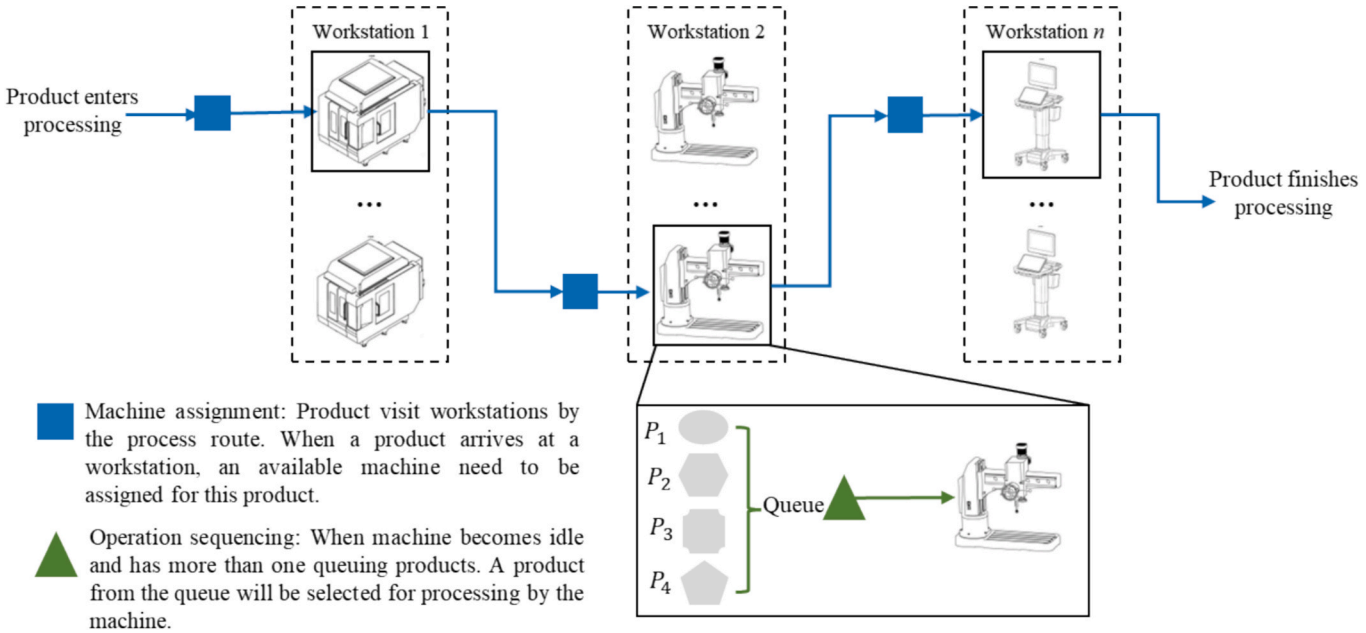


Fig. 1. Illustration of MA and OS.

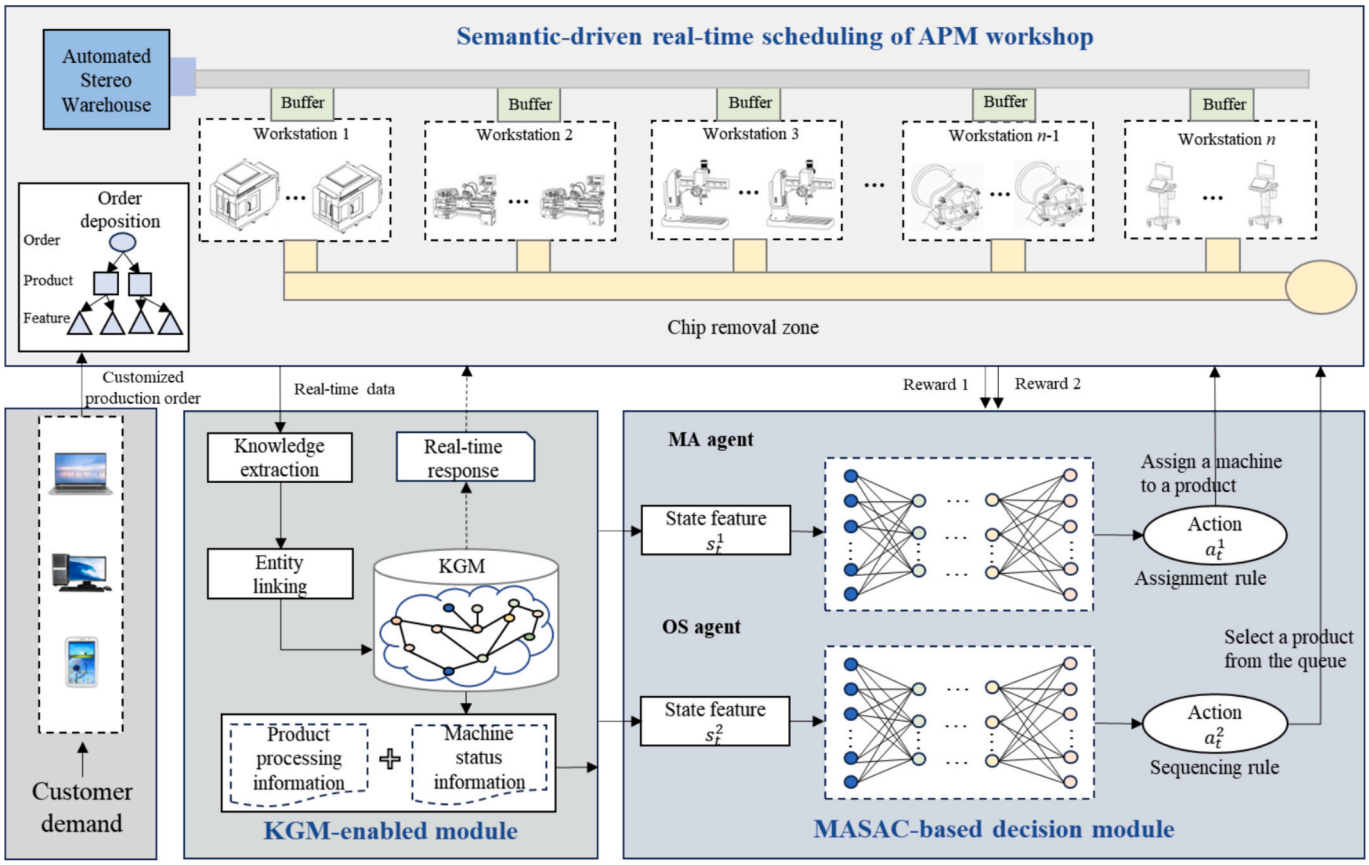


Fig. 2. Overall framework of the SRSA.

$$(S_{jks}, C_{jks}) \cap (S_{j'k's}, C_{j'k's}) = \phi \tag{6}$$

$$(C_{j1s} - t_{j1s} - A_j)X_{j1s} \geq 0 \tag{7}$$

$$(C_{jks} - t_{jks} - (FT_{rs} + rt_{rs}))X_{jks} \leq 0 \tag{8}$$

$$(C_{jks} - FT_{rs})X_{jks} \geq 0 \tag{9}$$

$$(C_{jks} - S_{jks} - t_{jks} - rt_{rs})X_{jks} = 0 \tag{10}$$

Where Eq. (2) states that the start time of each operation must be earlier than its completion time. Eq. (3) ensures that each operation can only be processed on one machine. Eq. (4) and (5) establish the priority constraints for successive operations. Eq. (6) guarantees that each machine can only handle one operation at a time. Eq. (7) specifies that each operation can only start after the product arrival time. Eq. (8) to (10) indicate that if an operation being processed is interrupted by a machine failure, it should be resumed on the same machine after repairs are completed.

4. Methodology: Proposed SRSA

The proposed SRSA has two major improvements. First, it leverages a knowledge graph to decouple manufacturing-related concepts. A KGM is constructed to integrate product, operation, and resource information, providing agents with real-time workshop status information during the decision-making process. In addition, logical rule-based response strategies are embedded within the KGM to enable prompt responses to dynamic disturbances. This integration significantly enhances the system's ability to handle disturbances, based on a comprehensive understanding of the production environment. Second, the state spaces, action spaces, reward functions, and asynchronous transitions are specifically designed for MA and OS agents. A MASAC algorithm is developed, in which these agents are trained to make effective scheduling decisions. As a result, the well-trained agents can realize real-time scheduling in the APM workshop. The appropriate dispatching rules can be adaptively selected and executed based on the production states provided by the KGM. The combination of the KGM and MASAC establishes a more robust and context-aware scheduling system that is well-suited to dynamic environments characterized by frequent disturbances.

4.1. Overall framework of the SRSA

The overall framework of the proposed approach is depicted in Fig. 2 and consists of two main modules: the KGM-enabled module and the MASAC-based decision module. During the production execution phase, upon receiving a customized production order, the manufacturing system decomposes the order to identify the specific type of product to be processed. The relevant production data for the product is then generated and transmitted to the knowledge graph module in real-time. Through the knowledge extraction method, entities related to processing information, such as product name, due date, and priority, are extracted from the production data and systematically linked to the constructed KGM. This allows processing information, such as available machines and auxiliary tools, to be matched with the product based on the KGM. Moreover, in the event of disturbances, the logical rules embedded within the KGM are triggered when specific conditions are met, facilitating a real-time response to dynamic disturbances.

In the MASAC-based decision module, the MA and OS agents are developed to select appropriate rules under different production states to complete machine assignment and operation sequencing, respectively. These agents are continuously trained and refined in the scheduling process. At the decision time, production states are derived from machine status information and product processing information

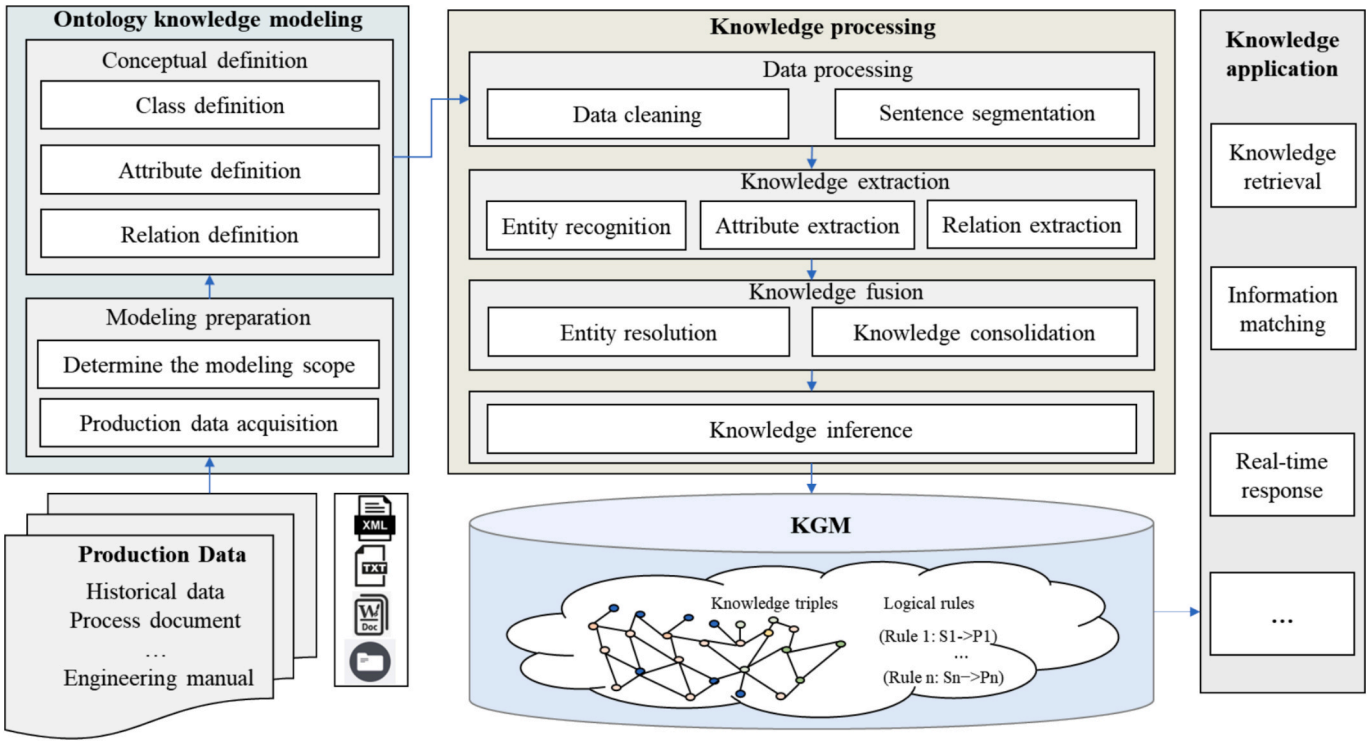


Fig. 3. Construction process of KGM.

provided by the KGM, allowing the agents to generate scheduling actions based on the current state. The selected actions from the MA and OS agents are then fed back to the workshop to guide real-time scheduling in the APM workshop. Concurrently, the agents receive feedback from the production environment in the form of rewards, reflecting the effectiveness of the selected rules.

4.2. Knowledge graph modeling for the APM workshop

To transform the multi-source and heterogeneous production data in the workshop into structured knowledge, the KGM for the APM workshop is constructed to integrate production information in a unified manner. The construction process, illustrated in Fig. 3, consists of two main parts: ontology knowledge modeling and knowledge processing. In the first part, corresponding classes, attributes, and semantic relationships are defined to establish a knowledge framework for the workshop. In the second part, the KGM is constructed following a series of steps, including knowledge extraction, fusion, and inference.

When the KGM is constructed, it can be applied to practical production scenarios. For example, manufacturing knowledge, such as product processing requirements, can be retrieved on demand. Additionally, processing information, including the available machines and auxiliary tools required for each operation, can be accurately matched to the products. Moreover, the response strategy within KGM for disturbances enables context-aware scheduling, allowing for effective responses to various disturbances.

4.2.1. Modeling for manufacturing process

The modeling scope focuses on the manufacturing process within the APM workshop. In the manufacturing process, various production data is generated at different stages. To model this process, historical data, process documents, and engineering manuals are collected from the workshop. A common glossary of terms is then derived from the production data to capture manufacturing-related concepts specific to the workshop. Using this glossary, corresponding classes, attributes, and semantic relationships are defined.

(1) Conceptual definition of manufacturing elements.

Based on the workshop manufacturing-related concepts collected from the production data, the classes can be classified into three types: machine, product, and operation, which are defined as follows.

Definition 1. (*The ontology of the machine concept can be formally defined as* $OT_M = < T_M, A_M, R_M >$*, which is described as follows.*) T_M *is the term abstraction related to the machine concept, such as turning machine, machining equipment, etc.* $T_M = [T_{M_1}, \cdots, T_{M_s}]$*, where* T_{M_s} *represents the term for a machine in the workshop.*

A_M is the attribute abstraction of the machine, and $A_M = \{FunI, BasI, StatI\}$. Specifically, *FunI* is the functional information of the machine, such as processing precision and positioning accuracy. *BasI* is the basic information of the machine, such as machine type, machine number, and machine location. *StatI* is the state information of the machine, such as queue size, available time, etc. The functional and basic information of the machine is static, but the state information of the machine is dynamic. In addition, the state information is constantly changing based on real-time data.

R_M is the relation abstraction of the machine. The relationship includes "has_attribute", "belong_to", "has_tool". The "has_attribute" is defined based on the attributes in A_M. The "belong_to" is defined because each machine belongs to a workstation in the APM workshop. The "has_tool" is defined because the machine is equipped with auxiliary tools such as fixture and knife. In manufacturing processing, the triplet < machine concept entity, attribute relationship, specific attribute > can represent the state information of the machine, such as < turning machine, has_attribute, available time>.

Definition 2. (*The ontology of product concept can be formally defined as* $OT_P = < T_P, A_P, R_P >$*, which are described as follows.*) T_P *is the term abstraction related to the product, such as plate and shaft.* $T_P = [T_{P_1}, \cdots, T_{P_j}]$*, where* T_{P_j} *represents the term for a product in the workshop.*

A_P is the attribute abstraction of the product, and $A_P = \{FeaI, ProT\}$, where *FeaI* is the feature information of the product, such as inner hole, groove, and outer surface. *ProT* is the production time parameters of the

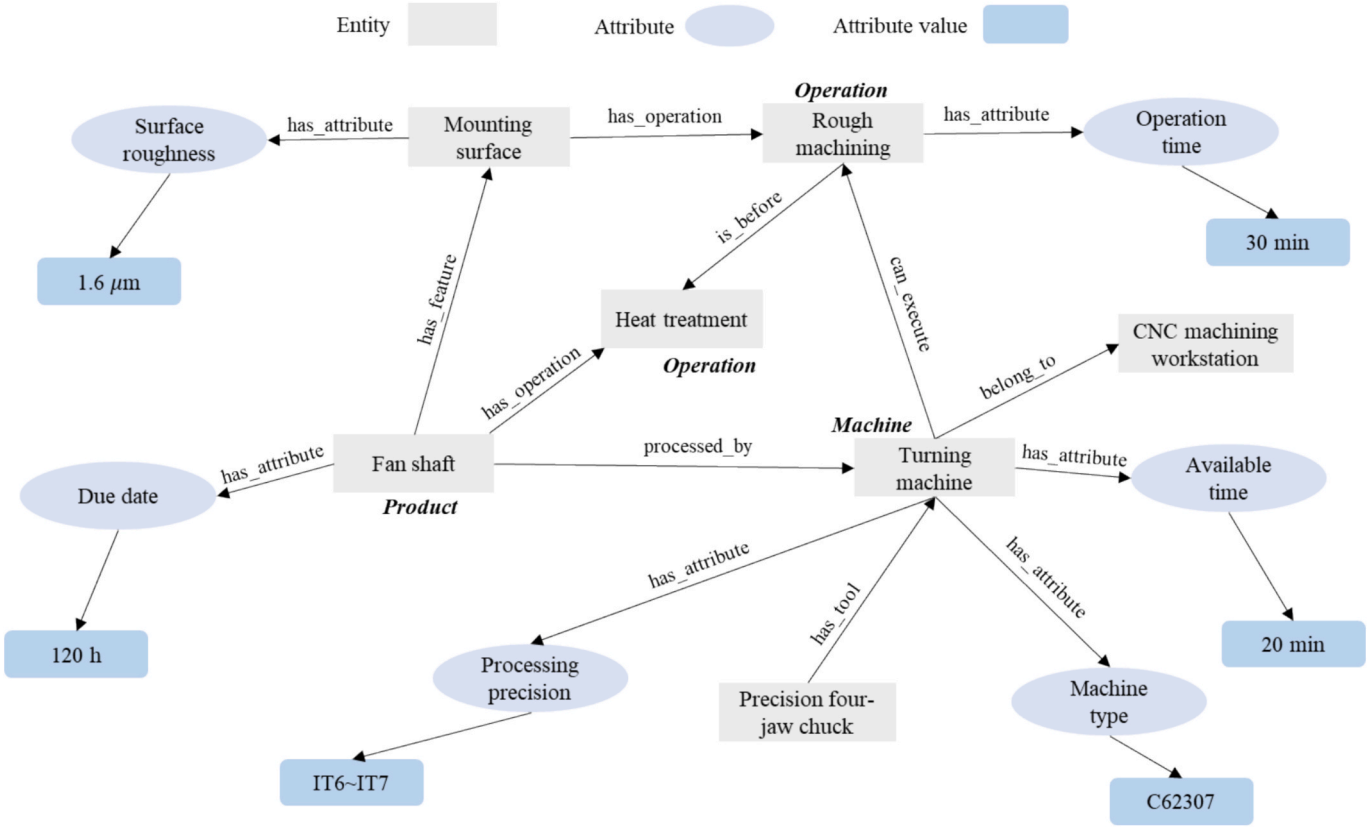


Fig. 4. Illustration of the relationship.

product such as the due date, slack time, etc. Moreover, the feature information is characterized by the surface roughness, size, length, etc.

R_P is the relation abstraction of the product, which contains the relationship "has_feature", and "has_attribute" based on the attributes in A_P. The triplet < product concept entity, feature relationship, specific feature > can represent the feature information of the product. For example, <fan shaft, has_feature, inner hole > means that the product "fan shaft" has the feature of inner hole.

Definition 3. (*The ontology of the operation can be formally defined as* $OT_O = < T_O, A_O, R_O >$*, which are described as follows.*) T_O *is the term abstraction related to the operation, such as precision machining and sandblasting.* $T_O = [T_{O_1}, \cdots, T_{O_k}]$*, where* T_{O_k} *represents the term for an operation required by the product in the workshop.*

A_O is the attribute abstraction of the operation concept. $A_O = \{OperI\}$, where *OperI* is the processing information of the operation, such as the operation time, start time, and completion time of an operation. This information is not static, but changes dynamically for different products in the manufacturing process.

R_O is the relation abstraction of the operation, which includes the relationship "has_attribute" based on the attributes in A_O and the relationship "is_before" based on the sequences of operations. The triplet < operation concept entity, attribute relationship, specific attribute > can represent the processing information of the operation, such as < precision machining, has_attribute, operation time > . Moreover, the relationship "is_before" characterizes the process route for producing a specific product, which also implicitly implies the temporal constraints of precedence order.

In addition, there are interrelated relationships between machines, products, and operations. These include the relationships "processed_by", "can_execute", and "has_operation". The "processed_by" relationship indicates which machine is capable of processing a

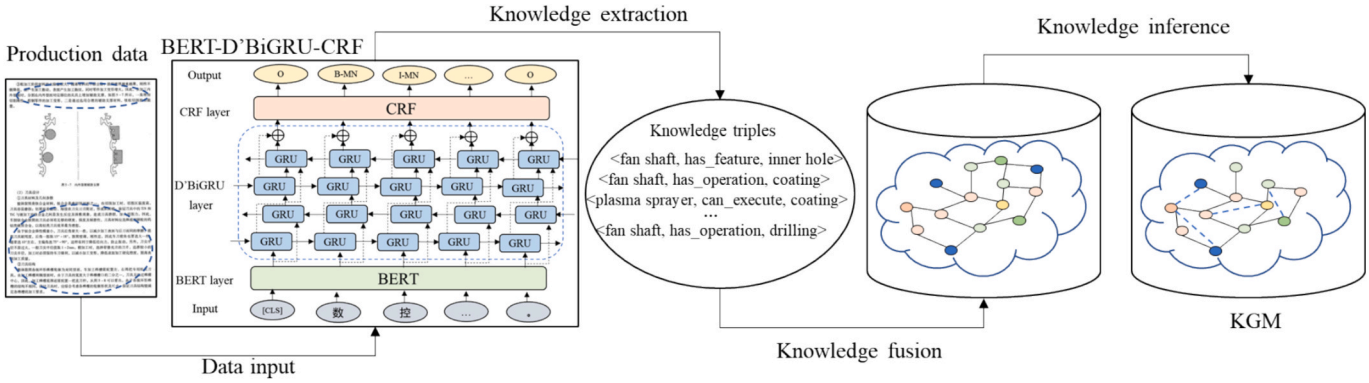


Fig. 5. Steps of knowledge processing.

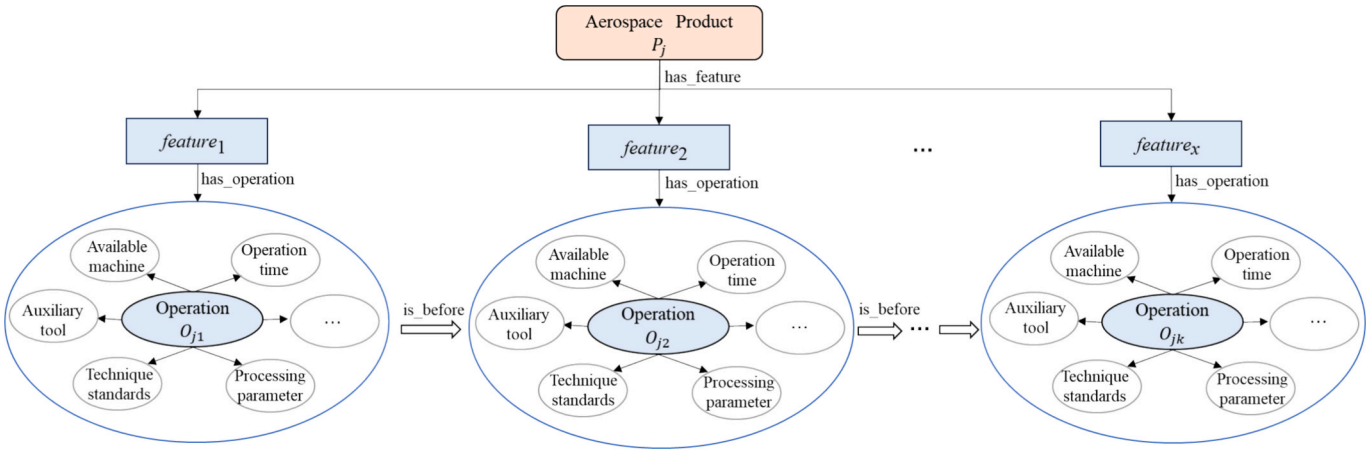


Fig. 6. Illustration of product processing relationship network.

particular product. The "can_execute" relationship specifies the machine that can execute a specific operation. Finally, the "has_operation" relationship outlines the operations required for a product (feature). Based on these definitions, a general example illustrating these relationships is shown in Fig. 4.

(2) Formation of the KGM of product relationship network.

The KGM can be constructed in a stepwise manner based on ontology modeling, as illustrated in Fig. 5. The process begins with the cleaning and segmentation of various forms of production data. Following this, entities, relationships, and attributes are jointly extracted from the processed data using the BERT-D'BiGRU-CRF method. In this method, the bidirectional encoder representations from transformers (BERT) layer captures the underlying semantic features of the input sequence, providing an embedded representation of entities through pretraining and fine-tuning (Qin, Zhao, & Liu, 2021). A double bidirectional gated recurrent unit (D'BiGRU) layer is employed to recognize long-term dependencies in text sequences, while the conditional random field (CRF) layer learns constraint information on label predictions (Xu & Dang, 2023).

Specifically, the BERT-D'BiGRU-CRF model is used to recognize 11 categories of entities: machine name (MN), machine functional information (FunI), machine basic information (BasI), machine state information (StatI), auxiliary tool (AT), product name (PN), product feature (FeaI), product time parameter (ProT), operation name (ON), operational information (OperI), and attribute value (AV). This model can accurately identify entities and attributive entities within the production data. Furthermore, by analyzing sentence syntax, relationships between entities are identified based on their positions within the sentence. The nearest-neighbor rule is applied to determine relationships, where the nearest entity of the same relation type is paired to form triples. For example, if both the PN and FeaI appear in a sentence, the predefined relation "has_feature" allows the model to generate a triple such as "fan shaft-has_feature-inner hole." Similarly, when a product name (PN) and a product time parameter (ProT) appear together, the predefined relation "has_attribute" facilitates the creation of a triple such as "fan shaft-has_attribute-due date," while also recognizing the attribute value of the due date in the same sentence. This approach effectively transforms the joint entity-relation extraction problem into a sequence-labeling task, enabling simultaneous recognition of entities and their corresponding relationships.

After knowledge extraction, redundant information is often present due to different terms referring to the same entity. For instance, "turning equipment" and "turning machine" correspond to the same entity. To improve the quality of the extracted knowledge, knowledge fusion is applied to remove redundant and ambiguous entities. An annotation rule is implemented to eliminate this redundancy and heterogeneity, which is described as follows. The entities with similar meanings are grouped into a unified entity set. Whenever an entity appears within this set, it is standardized to a single and consistent term. For example, if an entity occurs in the set ["turning equipment", "turning machine", ···], it is standardized to "turning machine". Following this step, the extracted knowledge is refined and represented as triples. However, these triples are initially independent and need consolidation. When triples share the same head or tail entity, such as in the cases of A = <a, d, e > and B = <a, b, c>, they can be imported into a Neo4j graph database to construct an interconnected knowledge network. In Neo4j, entities and their relationships are represented as nodes and edges, allowing different knowledge triples to be consolidated and linked.

The knowledge triple is the basic unit for constructing the KGM. However, there are some implicit relationships with certain preconditions between different entities. Therefore, the logical rules are used to realize the knowledge inference to identify the implicit relationship information of entities. The general form of logical rules is defined as follows.

$$X_1 \wedge X_2 \wedge X_3 \ldots \Longrightarrow X_N$$

Where X_i represents $r(\arg_1, \arg_2, \cdots, \arg_m)$, r represents the class, relation, or attribute, $\arg_1, \arg_2, \cdots, \arg_m$ denote the instance, attribute value, or variant. An example is provided below.

$\text{Product}(\text{product}_j) \bigwedge \text{has_feature}(\text{product}_j, \text{feature}_x) \bigwedge \text{has_operation}(\text{feature}_x, \text{operation}_k) \bigwedge \text{Operation}(\text{operation}_k) \Longrightarrow \text{has_operation}(\text{product}_j, \text{operation}_k)$.

Where feature_x is a feature of product_j, and product_j and operation_k are instances of product and operation classes, respectively. It can be inferred that if feature_x is associated with operation_k, then product_j is matched with operation_k. Based on this, a product can be decomposed into a set of operations through the "has_feature" and "has_operation" relationships. Furthermore, logical rules can be designed using empirical knowledge to enable product information matching and provide real-time responses to disturbances.

Definition 5. (*For the product produced in the workshop, the relationship network of processing information can be defined as* $\mathscr{g} = < P, RS, O, R, G >$, *and its meaning is described as follows.*) *P is the product to be produced. It consists of several processing features,* $\text{feature}_i \text{for} i = 1, \cdots, x$. *These features characterize the product after processing.*

RS is the resource entity set used to process the product, including the machine entity set {drilling machine, machining center, turning machine,···}, and tool entity set {knife, chuck, gauge, ···}.

O is the operation entity set required by the product, such as machining, heat treating, drilling, etc. Each operation corresponds to a processing feature.

R is the relationship associated with resources and operations for

product manufacturing.

G is the processing information of the product, represented by the triples $G = \{(h,r,t)\}$, where h is the head entity, t is the tail entity, and r is the relationship between entities. The processing of products involves multiple operations and requires a set of resources to execute these operations.

Based on the definitions above, the relationship network of processing information for a product can be developed using the KGM, as illustrated in Fig. 6. Each product is characterized by multiple processing features through the "has_feature" relationship, with each feature corresponding to a specific operation defined by the "has_operation" relationship. The sequence of operations required to produce the product is captured by the "is_before" relationship. Furthermore, each operation is associated with details such as operation time, technical standards, and processing parameters through the "has_attribute" relationship. To execute each operation, the available machines and auxiliary tools are matched based on knowledge triples and predefined logical rules. Specific examples of some logical rules for information matching are provided below.

(1) Information matching based on product characteristics:

Product(product_j) $\bigwedge$ has_feature(product_j, feature_x) $\bigwedge$ has_operation(feature_x, operation_k) $\bigwedge$ Machine(machine_{id}) $\bigwedge$ processing precision(feature_x, IT) $\bigwedge$ less than or equal(IT, m) $\bigwedge$ greater than(IT, n) $\Longrightarrow$ can_execute(machine_{id}, $\text{operation}_\text{k}$).

(2) Information matching based on tool requirements:

Product(product_j) $\bigwedge$ has_feature(product_j, feature_x) $\bigwedge$ has_operation(feature_x, $\text{operation}_\text{k}$) $\bigwedge$ Operation($\text{operation}_\text{k}$) $\bigwedge$ Machine (machine_{id}) $\bigwedge$ has_tool(machine_{id}, *toolA*) $\Longrightarrow$ can_execute(machine_{id}, $\text{operation}_\text{k}$).

(3) Information matching based on process route:

Product(product_j) $\bigwedge$ has_feature(product_j, feature_x) $\bigwedge$ has_operation(feature_x, $\text{operation}_\text{x}$) $\bigwedge$ has_feature (product_j, feature_y) $\bigwedge$ has_operation(feature_y, $\text{operation}_\text{y}$) $\bigwedge$ has_feature(product_j, feature_k) $\bigwedge$ has_operation(feature_k, $\text{operation}_\text{k}$) $\Longrightarrow$ is_before($\text{operation}_\text{x}$, $\text{operation}_\text{y}$) $\bigwedge$ is_before($\text{operation}_\text{y}$, $\text{operation}_\text{k}$).

In Rule (1), IT refers to the processing precision. According to this rule, feature information and processing precision are prerequisites. It can be inferred that machine_{id} can execute operation_k to process feature_x of product_j when the required processing precision is between levels n and m.

In Rule (2), if $\text{operation}_\text{k}$ for feature_x of product_j requires *toolA*, and machine_{id} is equipped with such a tool, then machine_{id} is capable of executing $\text{operation}_\text{k}$.

In Rule (3), the process route of product_j is sequentially organized. If product_j involves the features of feature_x, feature_y, and feature_k, and the corresponding operations are operation_x, operation_y, and operation_k, the processing route is as follows: $\text{operation}_x \rightarrow \text{operation}_y \rightarrow \text{operation}_\text{k}$.

4.2.2. Modeling for dynamic disturbances

Dynamic disturbances in APM workshop environment, such as machine failures, emergency orders, and random product arrivals, pose significant uncertainties to maintaining efficient production schedules. These disturbances must be accurately represented and modeled in a knowledge graph to enable real-time decision-making.

(1) Conceptual definition of dynamic disturbances.

The ontology of dynamic disturbances can be defined as $OT_D = < T_D, A_D, R_D >$, which is described as follows.

T_D denotes the type of disturbance events, such as machine failure, emergency order, random product arrival. $T_D = [T_{D_1}, \cdots, T_{D_z}]$, where T_{D_z} signifies the term for a specific type of disturbance.

A_D refers to the attributes of the disturbance, such as failure time, recovery time, priority level, arrival time, and so on. These attributes provide crucial contextual information about the nature of each disturbance, enabling more accurate modeling of its impact on the manufacturing process. For example, failure time and recovery time are essential for estimating the duration of a machine failure.

R_D is the relation abstraction of the impact that disturbances have on production scheduling. The impact of dynamic disturbances on production scheduling can be captured through their relationships with various manufacturing elements, such as machines, operations, and products. These relationships include "affect", "interrupt", and "reschedule". The "affect" indicates that an operation can be affected when a machine fails. The "interrupt" signifies that a machine stops working during a failure. The "reschedule" denotes the need to reschedule a product when disturbances occur.

With the above formal representation, the disturbance knowledge graph can be manually constructed and integrated into KGM based on the extensive experience of experts in handling disturbance events during the production process. Consequently, knowledge graphs can effectively represent and manage dynamic disturbances in the production workshop, resulting in a more flexible and efficient scheduling process.

(2) Response strategy based on logical inference.

Logical rules are essential for deriving implicit relationships and triggering response strategies in the face of dynamic disturbances. For example, if a machine failure occurs, the knowledge graph can infer which operations are affected and recommend alternative resources or reschedule the product. These inferences are based on predefined logical rules that consider the relationships between machines, products, operations, and the impact of disturbances. Specific examples are as follows.

(1) Disturbance(*machine failure*) $\bigwedge$ Machine(machine_s) $\bigwedge$ interrupt (machine_s, true) $\bigwedge$ affect(*machine failure*, operation_k) $\bigwedge$ belong_to (machine_s, $\text{workstation}_\text{i}$) $\bigwedge$ has_attibute(machine_s, *queue size*, m) $\bigwedge$ greater than(m, 0) $\bigwedge$ Product(product_b) $\Longrightarrow$ Reschedule(product_b, true).

(2) Disturbance(*emergency order*) $\bigwedge$ Product(product_j) $\bigwedge$ has_attibute(product_j, *priority*, *high*) $\bigwedge$ has_operation(product_j, operation_k) $\bigwedge$ Machine(machine_x) $\bigwedge$ can_execute(machine_x, operation_k) $\bigwedge$ has_attibute(operation_k, *operation time*, time_*x*) $\bigwedge$ can_execute(machine_s, operation_k) $\bigwedge$ has_attibute (operation_k, *operation time*, *time_s*) $\bigwedge$ less_-than(time_*s*, time_*x*) $\Longrightarrow$ Schedule(product_j, machine_s).

(3) Disturbance(*emergency order*) $\bigwedge$ Product(product_j) $\bigwedge$ has_attibute(product_j, *priority*, *high*) $\bigwedge$ tardy(product_j, true) $\bigwedge$ has_operation (product_j, operation_k) $\bigwedge$ can_execute(machine_s, operation_k) $\bigwedge$ Disturbance(*random_product_arrival*) $\bigwedge$ Product(product_k) $\bigwedge$ has_attibute (product_k, *priority*, *low*) $\bigwedge$ has_operation(product_k, operation_k) $\Longrightarrow$ processed_by(product_j, machine_s) $\bigwedge$ process (product_j, true) $\wedge\neg$ processed_by(product_k, machine_s).

Rule (1) states that if a machine failure occurs on machine_s and affects a specific operation_k, and if the queue size m of machine_s is greater than 0, then the product_b in the queue of machine_s should be rescheduled to an available machine within the same workstation.

Rule (2) specifies that a product_j in the emergency order has high priority and will be assigned to the machine_s with the shortest operation time.

Rule (3) states that if product_j from an emergency order has a higher processing priority than randomly arriving product_k and is tardy, machine_s will prioritize processing product_j to complete its operation_k.

These three response rules are designed to effectively manage dynamic disturbances by leveraging empirical knowledge in the scheduling process. Rule (1) is intended to improve the robustness and resilience of the scheduling process by ensuring that products are quickly rescheduled to available resources when machine failure occurs, thereby reducing delays and maintaining efficient production flow. Rule (2) ensures that high-priority emergency orders are handled with urgency and efficiency, minimizing the processing time while optimizing resource allocation. Rule (3) ensures that precedence is given to high-priority and time-sensitive products, particularly those at risk of being delayed. By ensuring their timely completion, the overall efficiency and

reliability of the production schedule are maintained. In combination, these rules facilitate a real-time scheduling process, supporting effective responses to unexpected disturbances in complex production environments.

By effectively modeling dynamic disturbances within the knowledge graph, workshops can achieve a more resilient and real-time scheduling process. The integration of logical rules ensures that the production process remains flexible and responsive to unexpected events. In the KGM, disturbance information is formalized, with disturbance entities and relationships linked to relevant machine, product, and operation entities. For instance, a machine failure entity is connected to the specific machine and the operations it impacts. This approach is feasible because disturbance events, such as machine failures or emergency orders, can be summarized from long-term production experience and historical data analysis. These insights can then be directly incorporated into the KGM.

4.3. Design of the MASAC

4.3.1. DRL preliminaries

RL is a powerful approach for solving sequential decision-making problems, which can be modeled as an MDP. An MDP is characterized by a five-tuple (S, A, P, γ, R). Specifically, S represents the state space, encompassing all possible states within the decision-making process. A denotes the action space, which includes all the actions that can be dynamically selected. P is the transition probability from the current state s_t to the next state s_{t+1} when an action $a \in A$ is executed. γ is the discount factor, and R is the reward function used to evaluate the quality of action execution.

Soft-actor-critic (SAC) (Haarnoja et al., 2018), one of the actor-critic methods, excels in solving decision-making problems. As a variant of soft Q-learning, SAC belongs to the class of maximum entropy reinforcement learning algorithms. Different from other DRL methods, the entropy of the policy ρ_π is added to the optimization objective of SAC, which is formulated as follows.

$$\pi^*_{max} = \underset{\pi}{\operatorname{argmax}} \sum_t E_{(s_t,a_t)\ \rho_\pi}[r(s_t,a_t) + \alpha \mathscr{H}(\pi(\bullet|s_t))] \tag{11}$$

Where α is the temperature coefficient, which measures the importance between the reward r and the maximum entropy term $\mathscr{H}$. Adding the term $\mathscr{H}$ enhances exploration during training, since the entropy obtained increases as the number of different actions executed by the agent increases. Then, the soft Bellman equation can be used to find the optimal policy π^*, which is formulated as follows.

$$Q(s_t,a_t) = r(s_t,a_t) + \gamma E_{s_{t+1}}[V(s_{t+1})] \tag{12}$$

Where $Q(s_t,a_t)$ is the Q-function, and $V(s_t) = E_{a_t\ \pi}[Q(s_t,a_t)] + \mathscr{H}(\pi(\bullet|s_t))$ is the soft state value function. The soft Q-value of the policy π can be estimated using Eq. (12) at each time step. For each Q-network, the loss function $L(w)$ is defined as follows.

$$L(w) = E_{(s_t,a_t)\ PL}[\frac{1}{2}(\min_{j=1,2} Q_{w_j}(s_t,a_t) - (r(s_t,a_t) + \gamma E_{s_{t+1}} V_{w^-}(s_{t+1})))^2] \tag{13}$$

Where PL is the experience replay pool. $V_{w^-}(s_{t+1})$ is the soft state value function parameterized by the Q-function parameter w, associated with Eq. (12). Then, the parameters of the policy network are updated by optimizing the following loss function $L(\theta)$.

$$L(\theta) = E_{s_t\ PL}[E_{a_t\ \pi_\theta}[\alpha \log \pi_\theta(a_t|s_t) - \min_{j=1,2} Q_{w_j}(s_t,a_t)]] \tag{14}$$

In SAC, the trade-off between exploration and exploitation is achieved through a maximum entropy reinforcement learning framework. Specifically, the policy is trained to maximize the expected return augmented by an entropy term, which encourages higher stochasticity in action selection. This entropy-augmented objective fosters more thorough exploration, particularly in high-dimensional or uncertain environments. To dynamically balance this trade-off, an automatically tuned temperature coefficient α is adopted, following the method proposed by Haarnoja et al. (2018). It adjusts the strength of the entropy term during training. A higher α leads to more exploration, while a lower α favors exploitation as training progresses.

Building on SAC, a multi-agent variant, MASAC, is developed to address the problem of APM real-time scheduling. MASAC inherits the entropy-augmented objective from SAC, enabling each agent to balance exploration and exploitation during training. This allows agents to learn efficient policies in dynamic production environments.

Table 4
Definition of state feature for MA agent.

No	Classification	Description of state feature
1–1	Machine-related	The available time of each machine at the current workstation
1–2		The queue size of each available machine at the current workstation
1–3	Operation-related	The operation time of the arrival product on each available machine at the current workstation
1–4		The sum of operation time of the arrival product on each available machine at the current workstation
1–5	Product-related	The due date of the arrival product waited for assignment at the current workstation
1–6		The slack time of the arrival product waited for assignment at the current workstation
1–7		The minimum slack time of the products that are being processed in other workstations and will arrive at the current workstation

4.3.2. Definition of crucial elements

In the scheduling of the APM workshop, there are two main tasks, MA and OS. Specifically, the scheduling involves selecting appropriate machines for each product and determining the sequence of products on each machine. With the increasing complexity of modern production systems, the development of multi-agent methods that can improve real-time responsiveness to systems has attracted growing attention. Thus, a MASAC algorithm has been developed in this study. In MASAC, the MA agent is responsible for machine assignment, while the OS agent handles operation sequencing. Each agent has its own state space, action space, and reward function. Both reward functions are designed to align with a shared global scheduling objective–to minimize total tardiness across all products. For each product, the scheduling decision is made in two stages: first by the MA agent to assign a machine, and then by the OS agent to determine the processing sequence on that machine. This sequential scheduling structure naturally establishes a cooperative relationship between the two agents. Although the agents do not share a critic network or communicate explicitly, their actions are naturally coordinated through the sequential decision-making process and the consistent optimization objective. This structure facilitates implicit cooperation between agents: the MA agent's assignment decisions affect the queue structure for the OS agent, and the OS agent's sequencing influences the effectiveness of future machine assignments made by the MA agent. As a result, the agents learn to cooperate through the environment and the joint influence of their actions on the global performance metric. The state space, action space, reward functions, and asynchronous transitions for these two agents are defined as follows.

1) State space.

The state comprehensively reflects the evolving production status of the workshop, thereby facilitating effective scheduling decisions. For the MA agent, the state includes machine status as well as product and operation information, as detailed in Table 4. During the scheduling process, the relevant information $\{StatI, ProT, OperI\}$ is obtained from KGM and abstracted to form the state feature vector s_t^1. This process ensures that the state representation captures a holistic view of the machine, product, and operation states. Similarly, for the OS agent, the

Table 5
Definition of state feature for OS agent.

No	Classification	Description of state feature
2–1	Operation-related	The sum of the remaining operation time of the queuing products
2–2		The average remaining operation time of the queuing products
2–3		The minimum remaining operation time of the queuing products
2–4		The maximum remaining operation time of the queuing products
2–5		The sum of the operation time of queuing products that are waiting for the next operation
2–6		The average operation time of queuing products that are waiting for the next operation
2–7		The minimum operation time of queuing products that are waiting for the next operation
2–8		The maximum operation time of queuing products that are waiting for the next operation
2–9		The ratio of maximum operation time to average operation time of queuing products
2–10		The ratio of minimum operation time to average operation time of queuing products
2–11	Product-related	The average value of the difference between current time and due date of queuing products
2–12		The minimum value of the difference between the current time and due date of queuing products
2–13		The number of products that are being processed in the system
2–14		The number of products that are waiting for processing in the queue
2–15		The average completion rate of the product in the system, where the completion rate of a product is the number of processed operations divided by the total operations.
2–16		The tardy rate of the products in the system
2–17		The minimum slack time of queuing products
2–18		The average slack time of queuing products
2–19		The maximum slack time of queuing products
2–20		The sum of the slack time of queuing products

Table 6
Definition of candidate rules for scheduling action.

Action	Classification	Candidate rule	Description
Action space 1	Machine assignment rule	Earliest available (EA)	Select the machine that is earliest available for processing
	Machine assignment rule	Shortest processing time (ST)	Select the machine that has the shortest operation time for the product.
	Machine assignment rule	Shortest total processing time (STP)	Select the machine that has the shortest total operation time of products that are waiting for processing.
Action space 2	Operation sequencing rule	Shortest processing time (SPT)	Select the product that has the shortest operation time.
	Operation sequencing rule	Earliest due date (EDD)	Select the product that has the earliest due date.
	Operation sequencing rule	Minimum slack time (MST)	Select the product that has the smallest slack time.
	Operation sequencing rule	Work in queue (WINQ)	Select the product that has the smallest sum of remaining operation time among the queuing products.
	Operation sequencing rule	Progression ratio (PR)	Select the product that has the smallest ratio of the interval between the current time and the due date to the remaining operation time.

state includes product and operation information, as outlined in Table 5. The KGM plays a critical role in deriving the information $\{ProT, OperI\}$, which is then abstracted to create the state feature vector s_t^2. It is evident that the KGM is essential in providing comprehensive workshop status information, including machine, product, and operation states.

2) Action space.

An action refers to a scheduling behavior that can be selected during the scheduling process. The action space for the MA agent is defined as $A^1 = [EA, ST, STP]$, where includes three rules for dispatching products to available machines. These rules are chosen to optimize machine utilization and reduce processing time by balancing machine loads and minimizing idle time. The action space for the OS agent is defined as $A^2 = [SPT, EDD, MST, WINQ, PR]$, containing five rules that determine which product in the queue can be processed by idle machines. These rules are chosen to effectively prioritize jobs based on factors like operation time, due dates, and workload distribution, ensuring a balance between meeting delivery deadlines and productivity. At each decision point, a feasible rule can be selected from the action space based on the observed state. Detailed information on the action spaces is provided in Table 6.

3) Reward function

The reward reflects the quality of the selected actions and, therefore, directly influences the policy changes. Given that the scheduling objective is to minimize total tardiness, the reward function should fully account for how scheduling decisions impact total tardiness. Accordingly, the reward functions for the MA and OS agents are designed as follows.

For the MA agent, when product P_j arrives at workstation W_i to perform the g-th operation O_{jg} at decision time t, the agent is responsible for selecting a rule to allocate an available machine for this product. Since not all products are scheduled at the decision time, the total tardiness is closely related to the slack time of each product. Minimizing total tardiness is effectively achieved by maintaining the slack time at each stage of production. Therefore, the reward function for the MA agent is designed based on the product slack time, as expressed below.

$$R_t = (ST_j^{t+1} - EST_j^{t+1})/L_i \tag{15}$$

$$ST_j^{t+1} = D_j - T_{current} - \sum_{k=g+1}^{q} \bar{t}_{jk} \tag{16}$$

$$EST_j^{t+1} = D_j - T_{current} - \sum_{k=g}^{q} \bar{t}_{jk} - \frac{1}{d} \sum_{M_s \in W_i} AV_s \tag{17}$$

Where $L_i = 50$ is a scaling factor to ensure R_t falls between -1 and 1, ST_j^{t+1} represents the actual slack time after the machine assignment, EST_j^{t+1} is the estimated slack time, $T_{current}$ is the current time, d is the number of available machines for performing O_{jg} in W_i, $\bar{t}_{jk} \in OperI$ is the average operation time of O_{jk} in a workstation, and $AV_s \in StatI$ is the available time of machine M_s. From Eq. (15), it is observed that the reward is calculated based on EST_j^{t+1} and ST_j^{t+1}. If ST_j^{t+1} is greater than EST_j^{t+1}, the product slack time is preserved after the machine assignment is performed, resulting in a larger reward R_t.

For the OS agent, they must prioritize the products queued in the buffer and select a rule to determine which product to process on an idle machine. Let the queued products for machine M_s be denoted as QP^s, and assume product P_j is selected for processing on M_s. If P_j has high priority, selecting P_j can reduce tardiness due to its tight due date. Conversely, if P_j has low priority, from a queueing perspective, the total tardiness is closely related to the queueing time of the products. When P_j is selected, its queuing time ends, while the queuing time of other products is extended by the operation time of P_j on M_s. Based on this, the reward function for the OS agents can be designed accordingly.

$$R_t = \begin{cases} 1, & \text{if the product has high priority} \\ \begin{cases} 0, & \text{otherwise} \\ 1, & \text{if} t_{js} \le \bar{t}_{QP^s} \end{cases} & \text{if the product has low priority} \end{cases} \quad (18)$$

Where $t_{js} \in OperI$ represents the operation time of the selected product P_j on machine M_s, and $\bar{t}_{QP^s} \in OperI$ denotes the average operation time of other queued products on the same machine M_s. A smaller t_{js} indicates that the queuing time for other products is shorter. Therefore, for a low-priority product, if t_{js} is smaller than $\bar{t}_{QP^s}$, the reward value is set to 1; otherwise, it is set to 0. For a high-priority product, the reward value is always set to 1.

4) Asynchronous transition

The asynchronous transitions of the MA and OS agents are described as follows. For the MA agent, the state s_t^1 is observed either upon the arrival of a new product or when a machine with queued products experiences a failure. In response, a scheduling action $a_t^1 \in A^1$ is then taken to dispatch the arriving product or a product from the failed machine's queue to an available machine. Subsequently, the reward r_t^1 is calculated based on the outcome of the action, and the next state s_{t+1}^1 is observed once the operation of the dispatched product is completed. For the OS agent, the state s_t^2 is observed when a machine becomes idle or is released after repair and there are pending jobs in its waiting queue. At this point, a scheduling action $a_t^2 \in A^2$ is taken to select a product to complete its operation. Following this, the reward r_t^2 is calculated, and the next state s_{t+1}^2 is observed upon the completion of the selected product's operation.

4.3.3. Training algorithm for the MASAC

A SAC-based training algorithm is developed to train the MA and OS agents, incorporating the response rules defined in Section 4.2.2, as shown in **Algorithm 1**. These rules are introduced to deterministically dispatch specific products, allocate specific machines, or trigger rescheduling events. They are activated only when their specific conditions are met and operate outside the agent's action space. Once triggered, the rule executes a predefined result, and no transition is stored in the replay buffer. As such, these rule-based decisions do not directly participate in policy learning or affect credit assignment. This mechanism effectively decouples the predefined knowledge-driven behavior from the data-driven reinforcement learning process. The main advantage of these response rules lies in handling production scenarios with predefined results, reducing the number of interactions needed for training and thus accelerating agent convergence. Furthermore, grounded in empirical knowledge, these rules provide optimal decisions for straightforward cases, ensuring efficient operation while enabling agents to focus on more uncertain and informative aspects of the environment. More details about the training process are outlined below.

The decision point t_1 for machine assignment occurs when a new product arrives or when a machine with queued products experiences a failure. In contrast, the decision point t_2 for operation sequencing is triggered when a machine becomes idle or is released after repair, provided there are pending jobs in its waiting queue. These decision points t_1 and t_2 are not necessarily synchronous. At the decision point, the MA and OS agents select a scheduling rule as their action from a predefined rule set. The environment triggers agent decisions only at specific system events, where the applicability of scheduling rules is guaranteed. Therefore, the action space at each decision point only includes valid rule choices based on the current state. This design prevents the generation of infeasible actions without requiring additional penalization or masking mechanisms. When a new product P_j with different processing priority arrives in the workshop, P_j is decomposed into a set of operations based on KGM. At the decision point t_1, if P_j has a high processing priority, the response rule (2) determines that a machine with the shortest operation time should be assigned. Otherwise, the current state $s_{t_1}^1$ abstracted from $\{StatI_{t_1}, ProT_{t_1}, OperI_{t_1}\}$ is observed, and a machine assignment rule $a_{t_1}^1$ is selected by the MA policy network $\pi_{\theta^1}(s_{t_1}^1)$. Consequently, machine $M_s \in M_{id}$ is chosen based on the execution of $a_{t_1}^1$. The reward $r_{t_1}^1$ is then calculated using the designed reward function, and the next state $s_{t_1+1}^1$ is entered. At the decision point t_2 for operation sequencing, if a product in the queue of M_s has a high processing priority and is already overdue, it is selected for processing based on a predefined response rule (3). Otherwise, the current state $s_{t_2}^2$ abstracted from $\{ProT_{t_2}, OperI_{t_2}\}$ is observed, and an operation sequencing rule $a_{t_2}^2$ is chosen by the OS policy network $\pi_{\theta^2}(s_{t_2}^2)$. A product in the queue of M_s is then selected for processing by executing $a_{t_2}^2$.

Algorithm 1 SAC-based Training Algorithm

1. Initialize the policy network π_{θ^1} and Q-networks $Q_{w_1^1}$ and $Q_{w_2^1}$ for MA agent. Parameterize target Q-networks $Q_{w_1^{1-}}$ and $Q_{w_2^{1-}}$ based on parameter replication $w_1^{1-} \leftarrow w_1^1$, $w_2^{1-} \leftarrow w_2^1$.
2. Initialize the policy network π_{θ^2} and Q-networks $Q_{w_1^2}$ and $Q_{w_2^2}$ for OS agent. Parameterize target Q-networks $Q_{w_1^{2-}}$ and $Q_{w_2^{2-}}$ based on parameter replication $w_1^{2-} \leftarrow w_1^2$, $w_2^{2-} \leftarrow w_2^2$.
3. Initialize the experience replay pool PL_1 and PL_2, respectively.
4. **For** $t = 1$: T **do** (t is the decision point, T is the terminal time when all the operations have been scheduled)
5. Product P_j randomly arrives in the workshop for processing. Decompose P_j into a set operations O_{jk}, and match each operation with available machines $\{M_{id}, i = 1, \cdots, n\}$ based on logical rules and triples in KGM.
6. If product P_j arrives at workstation W_i for processing O_{jk} at decision time t_1, activate the scheduling function of machine assignment:
7. If P_j has a high processing priority:
8. Assign a machine $M_s \in M_{id}$ with smallest operation time for P_j according to response rule (2)
9. Else:
10. Observe the current state $s_{t_1}^1 \leftarrow \{StatI_{t_1}, ProT_{t_1}, OperI_{t_1}\}$, select a machine assignment rule $a_{t_1}^1$ by π_{θ^1} based on the observed state $s_{t_1}^1$.
11. Execute $a_{t_1}^1$ to assign a machine $M_s \in M_{id}$ for P_j, and add P_j to the queue QP^s of machine M_s.
12. Calculate the reward $r_{t_1}^1$ according to Eq. (15). Then, enter the next state $s_{t_1+1}^1 \leftarrow \{StatI_{t_1+1}, ProT_{t_1+1}, OperI_{t_1+1}\}$. Collect the MA transition $(s_{t_1}^1, a_{t_1}^1, r_{t_1}^1, s_{t_1+1}^1)$ in PL_1.
13. If the machine M_s becomes idle at decision time t_2, activate the scheduling function of operation sequencing:
14. If P_j in the queue of M_s has a high processing priority and is already overdue:
15. P_j is selected from the queue for performing O_{jk} on machine M_s according to response rule (3)
16. Else:
17. Observe the current state $s_{t_2}^2 \leftarrow \{ProT_{t_2}, OperI_{t_2}\}$, select an operation sequencing rule $a_{t_2}^2$ by π_{θ^2} based on the observed state $s_{t_2}^2$.
18. Execute $a_{t_2}^2$ to select a product from the queue for performing O_{jk} on machine M_s.
19. Calculate the reward $r_{t_2}^2$ according to Eq. (18). Then, enter the next state $s_{t_2+1}^2 \leftarrow \{ProT_{t_2+1}, OperI_{t_2+1}\}$.
Collect the OS transition $(s_{t_2}^2, a_{t_2}^2, r_{t_2}^2, s_{t_2+1}^2)$ in PL_2.
20. If the r-th breakdown of machine M_s occurs:
21. If there exists an operation O_{jk} being processed on M_s:
22. $C_{jks} \leftarrow C_{jks} + rt_{rs}$, $AV_s \leftarrow C_{jks}$
23. Else:
24. $AV_s \leftarrow FT_{rs} + rt_{rs}$
25. If there exist queued products on M_s:
26. Activate the rescheduling function according to the response rule (1), and assign an alternative machine within the same workstation for these products following steps (6)-(12).
27. Sample N transitions from PL_1 to update $Q_{w_1^1}$ and $Q_{w_2^1}$ according to Eq. (13), and update π_{θ^1} according to Eq. (14).
28. Synchronize the parameters of target Q-networks: $w_1^{1-} \leftarrow \tau^1 w_1^1 + (1-\tau^1) w_1^{1-}$, $w_2^{1-} \leftarrow \tau^1 w_2^1 + (1-\tau^1) w_2^{1-}$.
29. Sample N transitions from PL_2 to update $Q_{w_1^2}$ and $Q_{w_2^2}$ according to Eq. (13), and update π_{θ^2} according to Eq. (14).
30. Synchronize the parameters of target Q-networks: $w_1^{2-} \leftarrow \tau^2 w_1^2 + (1-\tau^2) w_1^{2-}$, $w_2^{2-} \leftarrow \tau^2 w_2^2 + (1-\tau^2) w_2^{2-}$.

Machine failure is also accounted for in the scheduling process. If a machine M_s fails, it undergoes immediate repair with a specified repair

Table 7
Hyperparameter grid for training algorithm.

Parameter	Tested range	Selected (MA)	Selected (OS)
Discount factor (γ)	{0.85, 0.9, 0.95}	0.95	0.95
Soft update parameter (τ)	{0.001, 0.005, 0.01}	0.005	0.005
Capacity of experience replay pool(P)	{512, 1024}	512	512
Periodic time units(C)	{3, 5, 10}	3	5
Batch experience size(N)	{32, 64, 128}	32	32
Learning rate (l_r)	{0.001, 0.005, 0.01}	0.01	0.01

time rt_{rs}. If an operation was being processed at the time of failure, this operation resumes on M_s upon repair completion. Moreover, if there are products queued on M_s, response rule (1) is triggered to reschedule these products to an available machine within the same workstation, following steps (6)-(12).

The policy network and Q-networks of the MA and OS agents are optimized using the stochastic gradient descent (SGD) method. A mini-batch size of N transitions is sampled from the experience replay pool to update the Q-networks following Eq. (13), while the policy networks are updated according to Eq. (14). The parameters of the target networks are periodically synchronized with those of the Q-networks every C time units. The network architectures for the MA and OS agents are designed based on their respective input and output dimensions. The input of the network is the state feature and the output is the scheduling action. The MA agent's network comprises an input layer with 15 units, hidden layers ($48 \times 32 \times 16 \times 8$), and an output layer with 3 units. The OS agent's network consists of an input layer with 20 units, hidden layers ($64 \times 48 \times 48 \times 36 \times 24 \times 12$), and an output layer with 5 units. The hyperparameters used in training are determined through an empirical grid-based search strategy. Specifically, a series of pilot experiments are

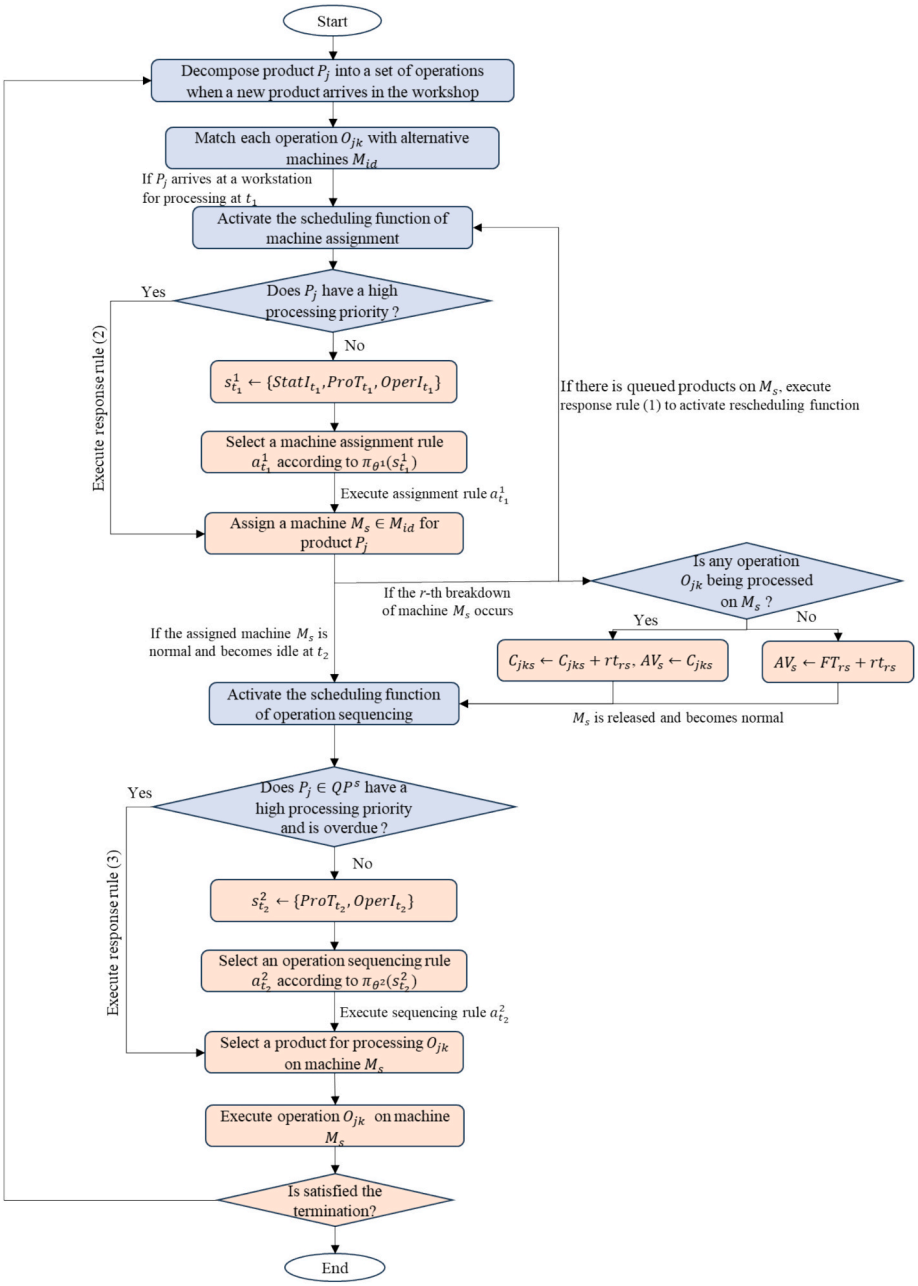


Fig. 7. Semantic-driven real-time scheduling process.

Table 8
The configuration of the workshop.

No	Workstation	Operation	Machine	Machine type
1	CNC machining workstation	Precision machining, semi-precision machining, etc.	Double column CNC vertical turning, milling, and grinding machining center	DKCK-ST80/ CTD40
2	Balancing workstation	Dynamic balancing	Vertical balancing machine	YYH-160A
3	Ultrasonic inspection workstation	Ultrasonic inspection	Water immersion ultrasonic inspection machine	CTB-4090
4	Shot peening workstation	Shot peening	Automatic shot peening machine	YD-1010
5	Coating workstation	Coating	Plasma Sprayer	DH-1080
6	Sand blowing workstation	Sand blowing	Automatic sand-blowing machine	XT-141/ GF-23
7	Corrosion inspection workstation	Corrosion inspection	Corrosion inspection machine	TD-5096
8	Heat treatment workstation	Heat treatment	Vacuum furnace	YL24

conducted to evaluate different parameter combinations, aiming to identify settings that ensure stable convergence and optimal performance. The selected hyperparameters and tested ranges are summarized in Table 7.

4.4. Semantic-driven real-time scheduling based on KGM and MASAC

The well-trained agents are utilized to guide the real-time scheduling of the APM workshop by integrating with the KGM. The semantic-driven real-time scheduling process is depicted in Fig. 7, where the blue components represent the KGM-enabled module and the red components represent the MASAC-based decision module. For each arrived product, the product is decomposed into a series of operations. The processing information for each operation is then matched based on the product relationship network within the KGM. Some logical rules are designed in KGM based on empirical knowledge to facilitate real-time responses to dynamic disturbances. At certain decision points, these rules can autonomously make scheduling decisions when specific prerequisites are met, such as assigning machines or selecting which queued product should be processed.

Furthermore, KGM can update the state information in the workshop in real-time and feed this information back to the agents. This allows the KGM to provide not only static production knowledge, but also dynamic updates on changes in the production environment. As a result, the agents receive accurate and real-time environmental information to make effective scheduling decisions. This integration enhances the agents' ability to perceive and understand the environment. By providing structured production knowledge, forming the state space, enabling real-time updates, supporting logical reasoning, and embedding empirical knowledge, the KGM significantly improves the system's capacity to handle dynamic disturbances in production scheduling.

5. Case study

The configuration of the APM workshop is detailed in Table 8. The workshop consists of eight workstations, each equipped with three machines. To model the production environment, a discrete event simulation tool, Simpy, is utilized. A Neo4j graph database is used to store the knowledge graph, with Python 3.8 employed to implement the scheduling method. The "py2neo" package is used to establish a connection between Neo4j and Python, allowing for the retrieval of product, machine, and operation information from the KGM to drive the scheduling process. The inference function is defined in Python to formalize logical rules, which are defined for information matching and response to disturbance. When the prerequisites are met, the function is triggered to perform logical reasoning and execute dynamic queries in conjunction with Neo4j. Experiments are then designed and conducted within the simulated production environment to validate both the role of the KGM and the scheduling performance of the proposed SRSA. All experiments are conducted on a system equipped with the Intel(R) Core (TM) i5-11300H CPU and 16 GB RAM.

Table 9
Product species information.

Product Type	shaft	journal	gearbox	Plate
Species	Fan shaft, flexible shaft, turboshaft, …	Fan journal, high-pressure turbine journal, compressor front journal, …	Gearbox with fairing stubs, folio ring gearbox, integral ring gearbox, …	Fan plate, turbine plate, compressor plate, …

The experiments involve the random generation of production orders, with each product randomly selected from four types: shaft, journal, gearbox, and plate. In APM, each product type is further subdivided into specific variants. For example, the "shaft" category includes the fan shaft, turboshaft, paddle shaft, and more. Similarly, the journal, gearbox, and plate categories each have multiple variants, as detailed in Table 9. Additionally, there are variations in product features, with specific operations and feature information provided in Table 10.

5.1. Experiment setup

For the experiment, 25 % of all products are designated as emergency orders with a high processing priority, while the remaining 75 % are assigned a low processing priority. These products arrive at the workshop in succession at random intervals. The interval time between two arrivals follows an exponential distribution, $Exp(8)$. The operation time for each operation is uniformly distributed between 5 and 40, represented as $U[5, 40]$. The due dates for high-priority products are calculated as $D_j = A_j + \bar{t}^* n^* U[1,\ 1.2]$, while those for lower-priority products are determined by $D_j = A_j + \bar{t}^* n^* U[1.5,\ 2]$, where $\bar{t} = 22.5$ is the mean operation time of an operation. During operation execution, the probability of machine failure is 2e-3, while in the idle state, this probability decreases to 3e-4. Furthermore, the repair time for a machine follows an exponential distribution, $Exp(30)$. These parameter settings define the default stochastic production environment, which remains fixed during both the training and testing processes. Under this setting, the model is trained for a total of 80,000 simulated time units, with actual training time approximately seven minutes. To further assess the robustness and generalization of the trained model, additional experiments under modified operating conditions are presented in Section 5.3.3.

During the training period, approximately 9800 products are processed. The tardiness of every 100 products is recorded during training, as shown in Fig. 8. It is observed that the tardiness for each batch of 100 products is significantly reduced as the training progresses. The reward curves of the OS and MA agents during training are presented in Fig. 9. For clarity, the OS agent's rewards are averaged over every 100 decision points, while the MA agent's rewards are averaged over every 400 decision points. This choice is due to the asynchronous nature of their decision-making processes, which results in different numbers of decision points along the same simulation timeline. The curves exhibit a generally stable trend with moderate fluctuations. Following an initial phase of rapid increase and variability, the rewards stabilize at relatively high values, indicating that the agents have effectively learned stable policies. Although some fluctuations persist throughout the training process, the absence of a downward trend suggests good convergence

Table 10
Operation and product feature information.

P_1(Fan shaft)		P_2(Fan plate)		P_3(Compressor rear journal)		...
Feature	Operation	Feature	Operation	Feature	Operation	...
Mounting surface	Rough machining	Fan plate	Ultrasonic inspection	Compressor rear journal	Ultrasonic inspection	...
Fan shaft	Heat treatment	Outer face	Rough machining	Typeface	Semi-precision machining	
Fan shaft	Ultrasonic inspection	Fan plate	Heat treatment	Compressor rear journal	Corrosion inspection	
Vent hole	Drilling	Inner face	Semi-precision machining	Compressor rear journal	Heat treatment	
Outer surface	Precision machining	Fan plate	Corrosion inspection	Typeface	Precision machining	
Spline	Grinding	Pressure hole	Drilling	Circular end tooth	Grinding	
Fan shaft	Balancing	Fan plate	Balancing	Journal surface	Sand blowing	
		Fan plate	Shot peening			

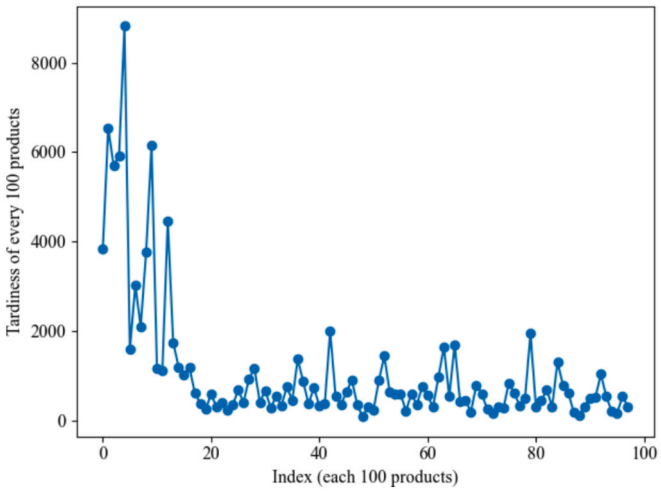


Fig. 8. Tardiness of each 100 products.

performance.

Numerical experiments are conducted on eight different cases, each with varying execution times set at 600, 900, 1200, 1500, 1800, 2100, 2400, and 2700 time units, respectively. The number of products for these cases is 74, 110, 147, 184, 221, 258, 295, and 331, respectively. To reduce the impact of randomness, each experiment is repeated 40 times under the same conditions. Across different cases, the trained agents are activated at each decision point to perform online decision-making. Based on the current state, the agents generate decisions within a few milliseconds, ensuring timely responses required by dynamic shop floor environments.

5.2. KGM-based information matching and state feature generation

The construction process of the KGM, as shown in Fig. 3, utilizes historical data, engineering manuals, and process documents. The resulting KGM is depicted in Fig. 10, encompassing key information from the APM workshop, such as machines, products, operations, tools, etc. Knowledge entities related to product, process, and resource information are represented in a unified and structured manner. In the lower-left corner of the figure, a product, the compressor front journal, is decomposed into five operations through the "has_operation" relationship, including balancing, stabilization, corrosion inspection, ultrasonic inspection, and fluorescence detection. In the lower-right corner, the stabilization operation is matched with the vacuum furnace machine via the "can_execute" relationship. The machine is further linked to the heat treatment workstation through the "belong_to" relationship. Based on this, the KGM generates an operation-workstation matching matrix, as shown in Fig. 11.

In addition, the KGM provides real-time machine status and product processing information by connecting to the Neo4j database via a Python driver and dynamically executing Cypher queries. This information is used to generate state features for the production environment. For example, as shown in Fig. 12, the data includes a machine status matrix and a processing information matrix. The machine status matrix reflects the status of each machine in a workstation, such as its available time and the number of queued products. Similarly, the processing information matrix, generated by the KGM, displays operation times for products on each machine within a workstation, along with product due dates. These matrices form the basis for generating state features s_t^1 and s_t^2. At decision points, agents can output feasible scheduling actions based on these state features. Therefore, the KGM plays a critical role in facilitating the timely generation of scheduling actions.

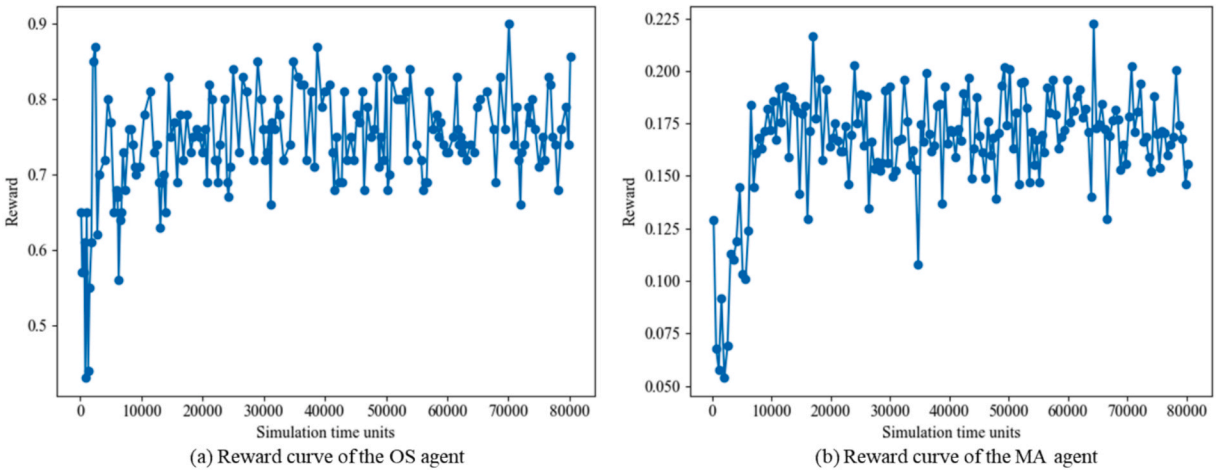


Fig. 9. Reward curves of agents during training.

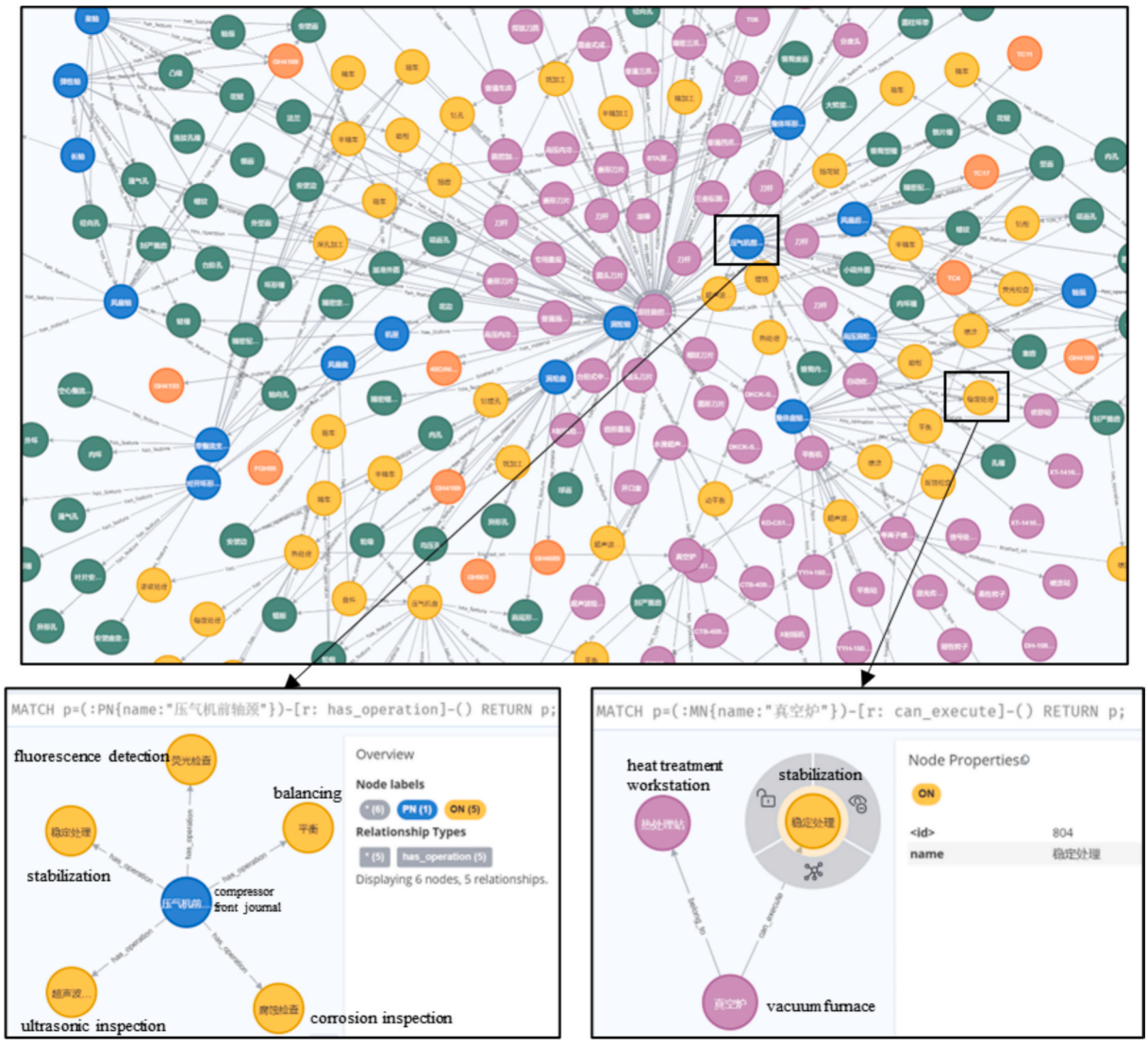


Fig. 10. Partial knowledge graph of the workshop.

	O_1	O_2	⋯	O_{q-1}	O_q
P_1	W_1	W_2	⋯	W_5	W_4
P_2	W_5	W_8	⋯	W_1	W_2
⋮	⋮	⋮	⋯	⋮	⋮
P_j	W_3	W_4	⋯	W_8	W_5

Fig. 11. Operation-workstation matching matrix.

5.3. *Numerical experiments for scheduling performance comparison*

To verify the effectiveness of the proposed method, we selected 15 basic composite dispatching rules to compare the scheduling performance with the SRSA. These rules are generated based on three rules from the MA agent's action space and five rules from the OS agent's action space. Additionally, we selected four machine assignment rules EA, ST, STP, and SQ, and three operation sequencing rules GP1 (Zhang, Yi, & Zhang, 2019), GP2 (Zhang et al., 2020), and ATC (Molina-Sánchez & González-Neira, 2016) from the literature to generate 12 composite rules for further performance comparison. Moreover, two popular DRL scheduling methods, DQN and Actor-Critic, are included to evaluate decision-making capabilities against the proposed method. It is noteworthy that *meta*-heuristic algorithms were not selected for comparison because they are unsuitable for real-time scheduling applications in a highly dynamic production environment.

The selected methods are evaluated under the same experimental conditions to ensure a fair comparison. To enhance the robustness of the results, a Wilcoxon signed-rank test is employed to determine if there are significant differences between methods at the 0.05 significance level. The comparison results between the proposed SRSA method and basic composite dispatching rules, as well as other rule-based and DRL-based methods, are presented in Tables 11 and 12, respectively. Furthermore, the robustness and generalization capability of the trained model are further evaluated under altered operating conditions, and the results are summarized in Table 13. In these tables, the average total tardiness across all repeated runs (Mean) is provided. The symbols "+", "=", "-" indicate that the proposed SRSA is better than, similar to, or worse than its competitor, respectively.

5.3.1. *Comparison with basic composite dispatching rules*

As shown in Table 11, the proposed SRSA outperforms other dispatching rules in all cases according to the performance metrics. The average objective value obtained by the SRSA is the smallest among the compared methods. Notably, no single dispatching rule yields the best results in every case. However, SRSA shows significant superiority over other methods, as confirmed by the results of the Wilcoxon signed-rank

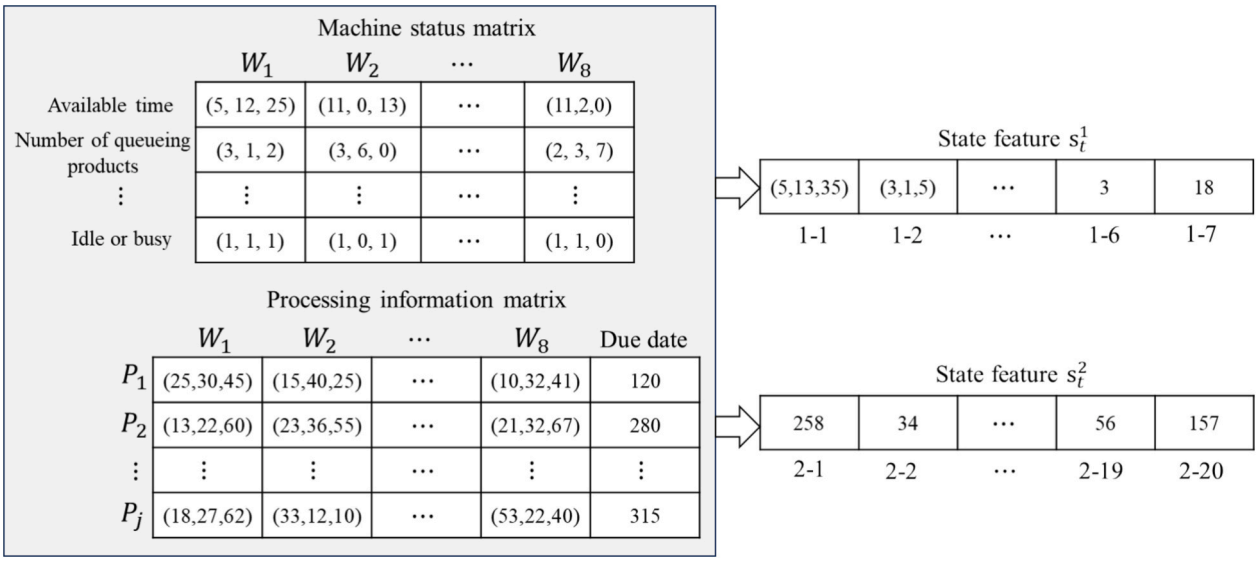


Fig. 12. Specific example of generating state features.

Table 11
Experimental results obtained by different methods in eight cases.

Case	I	II	III	IV	V	VI	VII	VIII
Metrics	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)
SRSA	2430	3679	5643	7295	8952	11,893	10,974	13,009
ST_EDD	3145 (+)	5743 (+)	9039 (+)	12,537 (+)	15,013 (+)	20,663 (+)	23,418 (+)	27,139 (+)
STP_EDD	2739 (+)	4736 (+)	7101 (+)	10,819 (+)	11,969 (+)	16,709 (+)	16,004 (+)	19,956 (+)
EA_EDD	2863 (+)	4596 (+)	6794 (+)	9824 (+)	11,727 (+)	16,491 (+)	16,206 (+)	18,626 (+)
ST_WINQ	4569 (+)	7555 (+)	11,403 (+)	14,923 (+)	18,676 (+)	24,289 (+)	26,510 (+)	31,085 (+)
STP_WINQ	3729 (+)	5946 (+)	8671 (+)	11,587 (+)	13,632 (+)	18,213 (+)	18,282 (+)	21,778 (+)
EA_WINQ	3592 (+)	5744 (+)	8405 (+)	11,144 (+)	12,465 (+)	17,436 (+)	17,348 (+)	19,869 (+)
ST_MST	3370 (+)	5766 (+)	9301 (+)	13,195 (+)	16,232 (+)	22,956 (+)	23,497 (+)	28,321 (+)
STP_MST	2949 (+)	4695 (+)	7428 (+)	10,504 (+)	12,326 (+)	16,530 (+)	16,420 (+)	19,356 (+)
EA_MST	2803 (+)	4548 (+)	6729 (+)	9790 (+)	11,151 (+)	15,690 (+)	15,526 (+)	18,414 (+)
ST_SPT	3315 (+)	5226 (+)	7857 (+)	10,938 (+)	12,865 (+)	16,645 (+)	18,482 (+)	20,180 (+)
STP_SPT	3013 (+)	4894 (+)	7027 (+)	9597 (+)	10,883 (+)	14,771 (+)	14,104 (+)	16,165 (+)
EA_SPT	2965 (+)	4607 (+)	6643 (+)	8998 (+)	10,094 (+)	14,041 (+)	13,327 (+)	15,404 (+)
ST_PR	3140 (+)	5727 (+)	8777 (+)	12,535 (+)	15,956 (+)	20,523 (+)	22,672 (+)	27,414 (+)
STP_PR	2907 (+)	4876 (+)	7107 (+)	10,292 (+)	12,153 (+)	16,499 (+)	16,625 (+)	18,886 (+)
EA_PR	2968 (+)	4657 (+)	6981 (+)	10,139 (+)	11,604 (+)	15,435 (+)	15,527 (+)	17,909 (+)

Table 12
Experimental results obtained by different methods in eight cases.

Case	I	II	III	IV	V	VI	VII	VIII
Metrics	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)
SRSA	2430	3679	5643	7295	8952	11,893	10,974	13,009
Actor-Critic	2817 (+)	4136 (+)	6691 (+)	9282 (+)	10,439 (+)	14,976 (+)	13,713 (+)	15,077 (+)
DQN	2653 (=)	3976 (+)	6511 (+)	9603 (+)	10,977 (+)	13,092 (+)	13,586 (+)	14,895 (+)
ST_GP1	3505 (+)	5842 (+)	9268 (+)	13,216 (+)	16,161 (+)	17,436 (+)	23,747 (+)	27,145 (+)
STP_GP1	3183 (+)	5113 (+)	8094 (+)	10,609 (+)	13,167 (+)	16,787 (+)	18,446 (+)	20,021 (+)
EA_GP1	3167 (+)	4949 (+)	7402 (+)	11,144 (+)	12,892 (+)	17,776 (+)	17,191 (+)	20,035 (+)
SQ_GP1	3109 (+)	5140 (+)	7783 (+)	11,062 (+)	12,838 (+)	21,015 (+)	17,915 (+)	21,975 (+)
ST_GP2	3719 (+)	6151 (+)	9552 (+)	12,973 (+)	16,306 (+)	16,763 (+)	23,138 (+)	26,890 (+)
STP_GP2	3169 (+)	5087 (+)	7486 (+)	10,403 (+)	12,374 (+)	16,145 (+)	16,970 (+)	19,392 (+)
EA_GP2	3349 (+)	5104 (+)	7294 (+)	10,468 (+)	12,146 (+)	17,162 (+)	16,192 (+)	19,049 (+)
SQ_GP2	3294 (+)	5498 (+)	7681 (+)	10,911 (+)	12,897 (+)	20,494 (+)	17,443 (+)	20,318 (+)
ST_ATC	3304 (+)	5730 (+)	8649 (+)	12,962 (+)	15,380 (+)	17,683 (+)	23,171 (+)	26,409 (+)
STP_ATC	2947 (+)	4940 (+)	7607 (+)	10,585 (+)	12,128 (+)	16,305 (+)	17,310 (+)	20,334 (+)
EA_ATC	2814 (+)	4601 (+)	6946 (+)	9805 (+)	11,660 (+)	17,264 (+)	16,474 (+)	18,755 (+)
SQ_ATC	2991 (+)	5006 (+)	7882 (+)	10,813 (+)	12,948 (+)	21,112 (+)	18,053 (+)	21,195 (+)

test. Therefore, at each decision point in the real-time scheduling process, the appropriate rule can be selected from the scheduling action space.

To evaluate the performance improvement of the SRSA over the dispatching rule, the average result B of the Mean metric from the dispatching rules in Table 11 is used as the reference standard. The improvement rate R of a method is defined as $R = (P - B)/B$, where P represents the scheduling result of the SRSA. Fig. 13 illustrates the

Table 13
Experimental results obtained by different methods in eight cases under altered operating conditions.

Case	I	II	III	IV	V	VI	VII	VIII
Metrics	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)	Mean(sign)
SRSA	3617	8097	15,430	22,397	31,744	43,397	53,802	67,096
ST_EDD	3872(+)	10005(+)	18037(+)	27570(+)	40158(+)	54830(+)	65248(+)	81419(+)
STP_EDD	4166(+)	9735(+)	18629(+)	27432(+)	40600(+)	54198(+)	68085(+)	86946(+)
EA_EDD	3913(+)	9146(+)	18248(+)	27005(+)	39727(+)	54105(+)	66898(+)	84446(+)
ST_WINQ	5445(+)	12567(+)	21935(+)	31647(+)	45741(+)	60633(+)	74142(+)	91796(+)
STP_WINQ	5383(+)	11689(+)	22099(+)	31536(+)	45948(+)	61944(+)	78241(+)	100296(+)
EA_WINQ	5326(+)	11449(+)	21906(+)	31420(+)	45732(+)	61609(+)	77053(+)	98459(+)
ST_MST	4439(+)	10505(+)	18899(+)	28751(+)	41142(+)	56227(+)	68092(+)	86048(+)
STP_MST	4494(+)	10293(+)	20144(+)	28917(+)	41866(+)	55827(+)	70342(+)	88685(+)
EA_MST	4291(+)	10210(+)	19583(+)	28657(+)	40880(+)	55775(+)	68820(+)	87389(+)
ST_SPT	4246(+)	9326(+)	16730(+)	24409(+)	34711(+)	46462(+)	56895(+)	68719(=)
STP_SPT	4656(+)	10399(+)	20531(+)	28964(+)	42613(+)	57526(+)	72297(+)	92814(+)
EA_SPT	4559(+)	10281(+)	19715(+)	29124(+)	41001(+)	55838(+)	70946(+)	90200(+)
ST_PR	4156(+)	10335(+)	18401(+)	26511(+)	39394(+)	52054(+)	63747(+)	78637(+)
STP_PR	4430(+)	10299(+)	19401(+)	28299(+)	40623(+)	54509(+)	67042(+)	86475(+)
EA_PR	4484(+)	9956(+)	18927(+)	27627(+)	40357(+)	53596(+)	66392(+)	83677(+)
ST_GP1	4266(+)	9676(+)	16823(+)	24952(+)	35172(+)	46845(+)	57508(+)	70330(+)
STP_GP1	4552(+)	9998(+)	18071(+)	26472(+)	37929(+)	50916(+)	62856(+)	79571(+)
EA_GP1	4284(+)	9659(+)	17852(+)	25657(+)	37218(+)	50247(+)	62448(+)	78631(+)
SQ_GP1	4328(+)	9697(+)	18616(+)	26508(+)	37585(+)	49903(+)	63077(+)	79733(+)
ST_GP2	4431(+)	10059(+)	17260(+)	25731(+)	35614(+)	48559(+)	60344(+)	72806(+)
STP_GP2	4669(+)	9897(+)	18328(+)	26982(+)	38415(+)	52098(+)	65346(+)	81980(+)
EA_GP2	4293(+)	9793(+)	18369(+)	26315(+)	37923(+)	51331(+)	63955(+)	81021(+)
SQ_GP2	4565(+)	10126(+)	18755(+)	26986(+)	38701(+)	51586(+)	65284(+)	83019(+)
ST_ATC	3994(+)	9674(+)	17435(+)	26106(+)	37561(+)	50302(+)	61445(+)	75069(+)
STP_ATC	4311(+)	9859(+)	18866(+)	27792(+)	40218(+)	53816(+)	67904(+)	86546(+)
EA_ATC	4150(+)	9552(+)	18261(+)	27018(+)	38784(+)	53081(+)	67209(+)	85333(+)
SQ_ATC	4290(+)	10178(+)	19185(+)	27475(+)	40455(+)	53665(+)	68540(+)	88875(+)
DQN	4040(+)	9434(+)	17251(+)	24303(+)	35513(+)	45954(+)	60690(+)	75210(+)
Actor-Critic	4417(+)	9838(+)	18915(+)	27054(+)	38777(+)	52985(+)	66256(+)	84081(+)

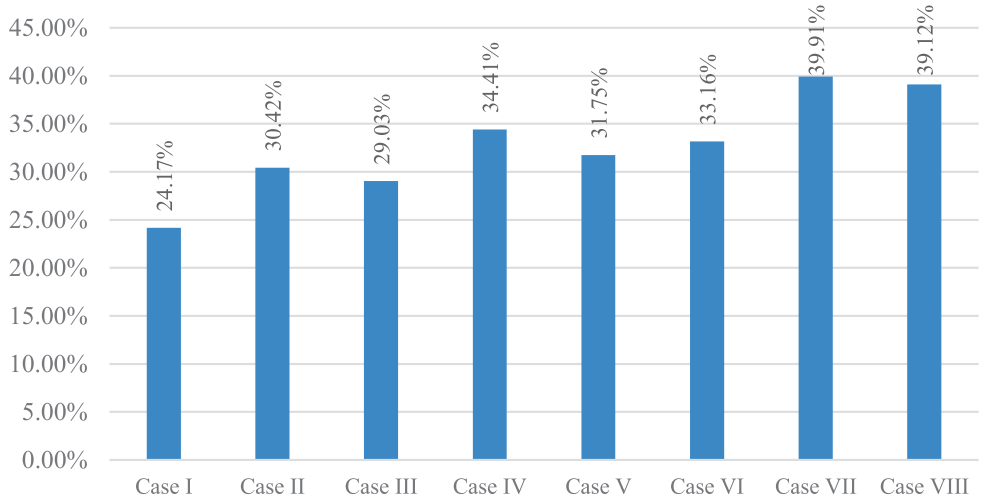


Fig. 13. Improvement rates of the SRSA.

performance improvement rates of the SRSA are plotted. It can be observed that the improvement rates of the SRSA, which are 24.17 %, 30.42 %, 29.03 %, 34.41 %, 31.75 %, 33.16 %, 39.91 %, and 39.12 % across various cases. Moreover, it is observed that the improvement rate of the SRSA increases as the number of products rises from case I to case VIII. This is because scheduling experience can be collected from real applications to periodically fine-tune the agents. Thus, the superiority of the proposed SRSA method is further validated from this perspective.

5.3.2. Comparison with other rule-based and DRL-based methods

According to Table 12, the proposed SRSA consistently outperforms the compared methods in terms of the Mean metric. The results of the Wilcoxon signed-rank test further confirm its significant superiority over other methods in most cases. To assess the robustness of the proposed approach, the winning rate is used as an additional performance measure, comparing SRSA with rule-based and DRL-based methods. The winning rate is defined as the number of runs in which a method achieves the minimum objective value, divided by the total number of runs. Fig. 14 shows the winning rates of SRSA and other methods, where SRSA achieves the highest winning rates of 32.5 %, 45 %, 57.5 %, 60 %, 52.5 %, 57.5 %, 72.5 %, and 62.5 % across all cases. Therefore, the decision-making ability of the proposed SRSA method is further demonstrated by the comparison results.

5.3.3. Robustness evaluation under variable condition

To further assess the robustness and generalization capability of the trained model, we have conducted additional tests under altered operating conditions. Given that variability in job arrival patterns and order

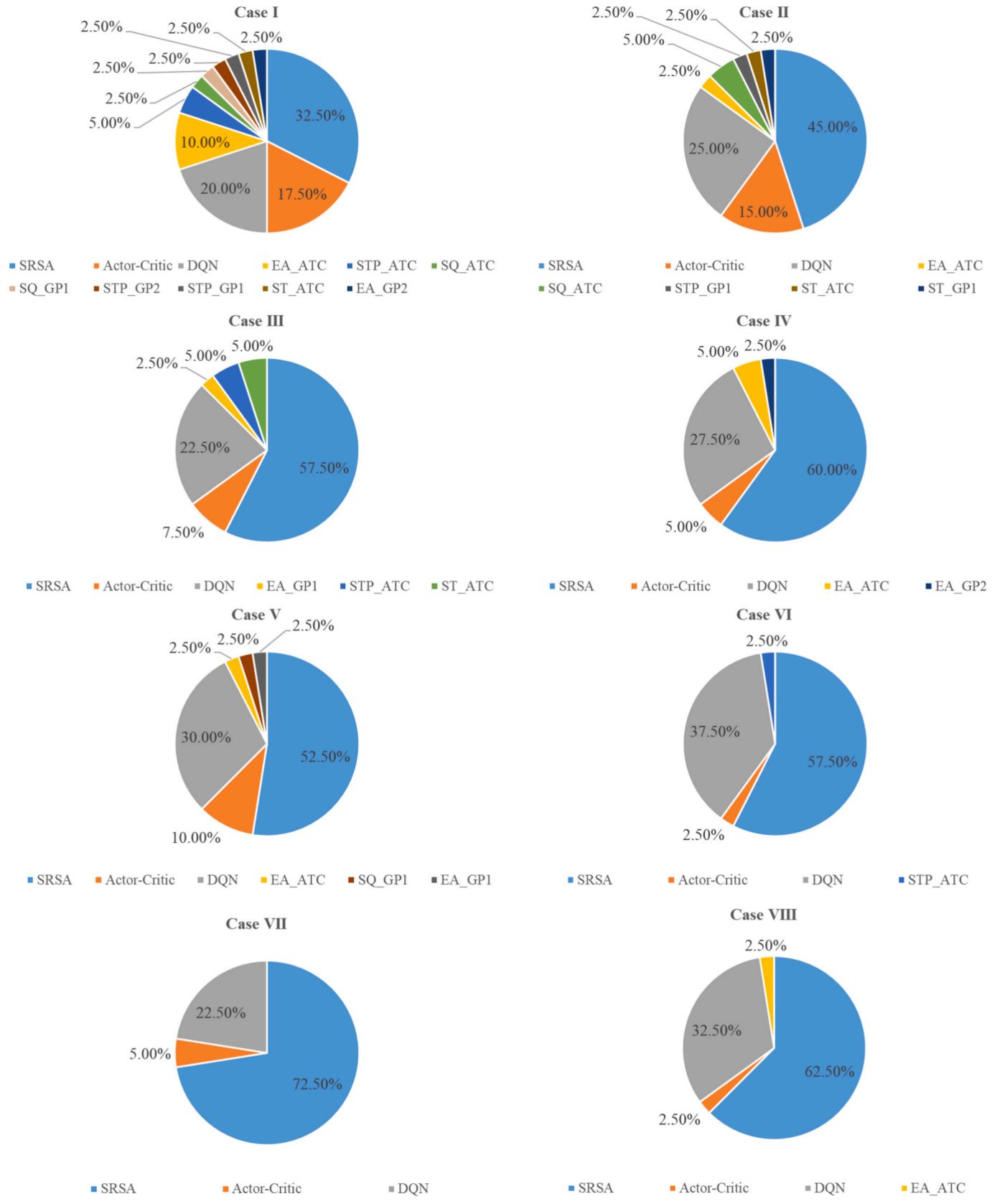


Fig. 14. Winning rates of different methods.

priorities typically constitute the primary sources of dynamic changes within the workshop, we focus on assessing robustness against changes in arrival patterns and priority distributions while keeping the machine failure probability constant. Specifically, the job arrival process is modified to follow a different exponential distribution, $Exp(6)$. In addition, the proportion of high-priority products is adjusted to 10 %, while the remaining 90 % are low-priority. The scheduling results of different methods are summarized in Table 13. As shown, the proposed method consistently outperforms the others in terms of numerical performance across all test cases. From a statistical perspective, it demonstrates comparable performance to ST_SPT in case VIII, while significantly surpassing other methods in all remaining cases. These findings further confirm the effectiveness and robustness of the trained model in dynamic and variable production conditions.

6. Conclusions

This study addresses the problem of real-time scheduling in APM, where frequent disturbances, such as machine failures, emergency orders, and random product arrivals, pose significant challenges. To handle the complexity inherent in APM, a semantic-driven real-time scheduling approach is proposed, which integrates a KGM with the MASAC.

The KGM is developed to integrate product, process, and resource information, providing semantic support for information matching and state feature generation. It incorporates the logical rule-based response strategy to improve responsiveness to dynamic disturbances. In MASAC, the state spaces, action spaces, reward functions, and asynchronous transitions are carefully designed. The agents are trained to dynamically select appropriate dispatching rules based on real-time production states. Experimental results demonstrate that SRSA outperforms other dispatching rules and DRL methods in a dynamic production environment.

In future work, this approach will be extended to handle multi-objective optimization problems, considering trade-offs among multiple performance indicators. We also intend to investigate the generalization and scalability of the system in more complex manufacturing environments with diverse products and equipment. To support larger-scale applications, we plan to explore more scalable agent architectures, such as hierarchical reinforcement learning.

CRediT authorship contribution statement

Chen Ding: Writing – review & editing, Writing – original draft, Software, Resources, Methodology, Formal analysis, Conceptualization. **Fei Qiao:** Project administration, Funding acquisition, Formal analysis. **Juan Liu:** Writing – review & editing, Investigation, Formal analysis. **Dongyuan Wang:** Writing – review & editing, Formal analysis. **Jiaxuan Shi:** Writing – review & editing, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (No. 62133011, 62373288, 62273260, and 62203336).

Data availability

The authors do not have permission to share data.

References

Chen, X., Li, J., Wang, Z., Chen, Q., et al. (2025). Optimizing dynamic flexible job shop scheduling using an evolutionary multi-task optimization framework and genetic programming. *IEEE Transactions on Evolutionary Computation*, 1–15.

Cong, Y., Yu, S., Chu, J., Huang, Y., et al. (2025). Enhancing novel product iteration: An integrated framework for heuristic ideation via interpretable conceptual design knowledge graph. *Advanced Engineering Informatics, 65*, Article 103131.

Dauzère-Pérès, S., Ding, J., Shen, L., & Tamssaouet, K. (2023). The flexible job shop scheduling problem: A review. *European Journal of Operational Research*, *314*(2), 409–432.

Destouet, C., Tlahig, H., Bettayeb, B., & Mazari, B. (2023). Flexible job shop scheduling problem under Industry 5.0: A survey on human reintegration, environmental consideration and resilience improvement. *Journal of Manufacturing Systems, 67*, 155–173.

Gu, W., Liu, S., Guo, Z., Yuan, M., et al. (2024). Dynamic scheduling mechanism for intelligent workshop with deep reinforcement learning method based on multi-agent system architecture. *Computers & Industrial Engineering*, *191*, 110115.

Guan, S., Cheng, X., Bai, L., Zhang, F., et al. (2023). What is event knowledge graph: A survey. *IEEE Transactions on Knowledge and Data Engineering, 99*, 1–20.

Gui, Y., Tang, D., Zhu, H., Zhang, Y., & Zhang, Z. (2023). Dynamic scheduling for flexible job shop using a deep reinforcement learning approach. *Computers & Industrial Engineering, 180*, Article 109255.

Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., et al. (2018). Soft Actor-Critic algorithms and applications. *Statistics*.

Jan, Z., Ahamed, F., Mayer, W., Patel, N., et al. (2023). Artificial intelligence for industry 4.0: Systematic review of applications, challenges, and opportunities. *Expert Systems with Applications, 216*, Article 119456.

Kong, S., Huang, X., Zhong, X., & Yang, M. (2024). Entity recognition method for airborne products metrological traceability knowledge graph construction. *Measurement, 225*, Article 114032.

Ladosz, P., Weng, L., Kim, M., & Oh, H. (2022). Exploration in deep reinforcement learning: A survey. *Information Fusion, 85*, 1–22.

Lavanya, V., & Saravanan, M. (2025). SARSA-based reinforcement learning framework for energy-aware and makespan-optimized workload scheduling in cloud computing. *Journal of Information Systems Engineering and Management, 10*, 198–209.

Li, D., & Tang, H. (2021). A semantic-level component-based scheduling method for customized manufacturing. *Robotics & Computer-Integrated Manufacturing, 71*, Article 102144.

Li, Y., Gu, W., Yuan, M., & Tang, Y. (2022a). Real-time data-driven dynamic scheduling for flexible job shop with insufficient transportation resources using hybrid deep Q network. *Robotics & Computer-Integrated Manufacturing, 74*, Article 102283.

Li, Z., Gao, L., Gao, Y., Li, X., & Li, H. (2022b). Zero-shot surface defect recognition with class knowledge graph. *Advanced Engineering Informatics, 54*, Article 101813.

Lim, K., Liu, Y., Chen, C., & Gu, X. (2024). Manufacturing resilience through disruption mitigation using attention-based consistently-attributed graph embedded decision support system. *Computers & Industrial Engineering, 197*, Article 110494.

Lu, S., Wang, Y., Kong, M., Wang, W., et al. (2024). A double Deep Q-Network framework for a flexible job shop scheduling problem with dynamic job arrivals and urgent job insertions. *Engineering Applications of Artificial Intelligence, 133*, Article 108487.

Luo, S. (2020). Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Applied Soft Computing, 91*, Article 106208.

Luo, S., Zhang, L., & Fan, Y. (2021). Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. *Computers & Industrial Engineering, 159*, Article 107489.

Molina-Sánchez, L., & González-Neira, E. (2016). GRASP to minimize total weighted tardiness in a permutation flow shop environment. *International Journal of Industrial Engineering Computations, 7*(1), 161–176.

Oukil, A., & El-Bouri, A. (2021). Ranking dispatching rules in multi-objective dynamic flow shop scheduling: A multi-faceted perspective. *International Journal of Production Research, 59*(2), 388–411.

Peng, C., Sheng, Y., Gui, W., Tang, Z., & Li, C. (2024). A rolling bearing fault diagnosis method based on multimodal knowledge graph. *IEEE Transactions on Industrial Informatics, 20*(11), 13047–13057.

Qin, Q., Zhao, S., & Liu, C. (2021). A BERT-BiGRU-CRF model for entity recognition of chinese electronic medical records. *Complexity, 2021*(3), 1–11.

Qiu, J., Liu, J., Peng, C., & Chen, Q. (2024). A novel predictive–reactive scheduling method for parallel batch processor lot-sizing and scheduling with sequence-dependent setup time. *Computers & Industrial Engineering, 189*, Article 109985.

Salama, S., Kaihara, T., Fujii, N., & Kokuryo, D. (2022). A novel feature selection for evolving compact dispatching rules using genetic programming for dynamic job shop scheduling. *International Journal of Production Research, 60*(13), 1–24.

Wang, Y. (2020). Adaptive job shop scheduling strategy based on weighted Q-learning algorithm. *Journal of Intelligent Manufacturing, 31*(2), 417–432.

Wu, X., & Yan, X. (2023). An improved Q learning algorithm to optimize green dynamic scheduling problem in a reentrant hybrid flow shop. *Journal of Mechanical Engineering, 59*(13), 246–259.

Wu, X., Yan, X., Guan, D., & Wei, M. (2024). A deep reinforcement learning model for dynamic job-shop scheduling problem with uncertain processing time. *Engineering Applications of Artificial Intelligence, 131*, 107790–107803.

Xiao, Y., Zheng, S., Leng, J., Gao, R., et al. (2024). An assembly process planning pipeline for industrial electronic equipment based on knowledge graph with bidirectional extracted knowledge from historical process documents. *Journal of Intelligent Manufacturing*, 1–21.

Xiong, H., Wang, H., Shi, S., & Chen, K. (2024). Comparison study of dispatching rules and heuristics for online scheduling of single machine scheduling problem with predicted release time jobs. *Expert Systems With Applications, 243*, Article 122752.

Xu, Z., & Dang, Y. (2023). Data-driven causal knowledge graph construction for root cause analysis in quality problem solving. *International Journal of Production Research, 61*(10), 3227–3245.

Zhang, F., Yi, M., & Zhang, M. (2019). A two-stage genetic programming hyper-heuristic approach with feature selection for dynamic flexible job shop scheduling. In *Proceedings of the Genetic and Evolutionary Computation Conference* (pp. 347–355).

Zhang, F., Mei, Y., Nguyen, S., & Zhang, M. (2020). Evolving scheduling heuristics via genetic programming with feature selection in dynamic flexible job shop scheduling. *IEEE Transactions on Cybernetics, 51*(4), 1797–1811.

Zhang, L., Feng, Y., Xiao, Q., Xu, Y., et al. (2023). Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times. *Journal of Manufacturing Systems, 71*, 257–273.

Zhang, L., Yan, Y., Yang, C., & Hu, Y. (2024). Dynamic flexible job-shop scheduling by multi-agent reinforcement learning with reward-shaping. *Advanced Engineering Informatics, 62*, Article 102872.

Zhao, Z., Zhang, M., Chen, J., Qu, T., et al. (2022). Digital twin-enabled dynamic spatial-temporal knowledge graph for production logistics resource allocation. *Computers & Industrial Engineering, 171*, Article 108454.